

國立中央大學
114 學年度 第一學期

軟體工程 期末書面報告

D 物件設計文件

寵物領養平台

組別:第十七組

組員:

114423001 陳柏甫

114423007 潘維晟

114423042 劉和媛

114423043 李嶧

目錄

1. 文件概述.....	7
1.1 目的.....	7
1.2 適用範圍.....	7
1.3 相關文件.....	7
2. 核心資料模型 (Data Models)	7
2.1 User (使用者)	7
2.2 Animal (動物).....	8
2.3 Application (領養申請).....	10
2.4 Shelter (收容所).....	11
2.5 MedicalRecord (醫療紀錄)	12
2.6 Notification (通知).....	13
2.7 Job (背景任務)	14
2.8 Attachment (附件)	15
2.9 AuditLog (稽核日誌)	16
3. REST API 詳細規格	17
3.1 Authentication APIs.....	17
3.2 Animals APIs	18
3.3 Applications APIs	22
3.4 Uploads APIs	24
3.5 Medical Records APIs	25
3.6 Shelters APIs.....	27
3.7 Notifications APIs.....	29
3.8 Jobs APIs	31
3.9 Users APIs	32

3.10 Admin APIs	33
4. 前端資料結構 (TypeScript Interfaces)	34
4.1 Animals API Types	34
4.2 Applications API Types	36
4.3 Medical Records API Types	37
4.4 Shelters API Types	38
4.5 Notifications API Types	38
4.6 Jobs API Types	39
4.7 Uploads API Types	40
5. 循序圖索引	41
5.1 動物瀏覽與搜尋	41
5.2 個人送養流程	49
5.3 領養申請流程	59
5.4 審核流程	61
5.5 收容所管理	67
5.6 醫療紀錄	71
5.7 通知系統	75
6. 物件互動說明	83
6.1 動物列表查詢流程	83
6.2 領養申請提交流程	85
6.3 檔案上傳流程	87
6. 物件互動說明	87
6.1 動物列表查詢流程	87
6.2 動物詳情檢視流程	89
6.3 領養申請提交流程	90

6.4 申請審核流程.....	93
6.5 個人送養發佈流程.....	95
6.6 收容所批次匯入流程.....	98
6.7 通知系統流程.....	101
7. 資料驗證規則.....	102
7.1 後端驗證 (Python/Flask)	102
7.2 前端驗證 (TypeScript/Vue)	103
8. 錯誤處理.....	104
8.1 HTTP 狀態碼規範.....	104
8.2 錯誤回應格式.....	104
9. 並發控制.....	105
9.1 Optimistic Locking	105
9.2 Idempotency (冪等性).....	105
10. 效能考量.....	106
10.1 資料庫查詢優化.....	106
10.2 快取策略.....	106
10.3 非同步處理.....	106
10. 效能考量.....	107
10.1 資料庫查詢優化.....	107
10.2 快取策略.....	108
10.3 圖片處理與 CDN	109
10.4 非同步處理.....	110
10.5 Rate Limiting (速率限制).....	111
11. 安全性考量.....	112
11.1 認證與授權.....	112

11.2 輸入驗證.....	113
11.3 檔案上傳安全.....	114
11.4 敏感資料處理.....	115
12. 測試策略.....	116
12.1 後端測試.....	116
12.2 前端測試.....	117
13. 部署架構.....	118
13.1 Docker Compose (開發環境).....	118
13.2 生產環境架構.....	119
附錄 A: 完整 API 端點列表	120
Authentication.....	120
Animals.....	120
Applications	121
Medical Records	121
Shelters.....	121
Notifications.....	122
Jobs	122
Uploads	122
Users	122
Admin	122
附錄 B: 循序圖完整列表.....	124
使用案例 1: 動物瀏覽與搜尋	124
使用案例 2: 個人送養流程	128
使用案例 3: 領養申請流程	133
使用案例 4: 審核流程	136

使用案例 5: 收容所管理	139
使用案例 6: 醫療紀錄	141
使用案例 7: 通知系統	143

1. 文件概述

1.1 目的

本文件描述系統的詳細物件設計，包括：- 資料模型結構與關聯 - API 端點詳細規格（輸入/輸出 JSON 格式） - 關鍵 Use Case 的循序圖 - 前後端資料流與互動細節

1.2 適用範圍

此文件適用於開發團隊進程式碼實作，包含前端（Vue 3 + TypeScript）與後端（Flask + Python）的物件層級設計。

1.3 相關文件

- 功能規格文件 (spec.md)
- 架構設計文件 (c_architecture.md)
- 需求分析文件 (b_requirementAnalysis.md)
- 循序圖 (sequence_diagram/)

2. 核心資料模型 (Data Models)

2.1 User (使用者)

檔案位置: backend/app/models/user.py

資料結構:

```
class UserRole(Enum):
    GENERAL_MEMBER = 'GENERAL_MEMBER' # 一般會員
    SHELTER_MEMBER = 'SHELTER_MEMBER' # 收容所會員
    ADMIN = 'ADMIN' # 管理員
```

```
class User(db.Model):
    user_id: int (PK, Auto)
    email: str (Unique, Not Null)
    username: str (Nullable)
    phone_number: str (Nullable)
    first_name: str (Nullable)
    last_name: str (Nullable)
    region: str (Nullable) # 地區/縣市
    address: JSON (Nullable)
    role: UserRole (Not Null, Default: GENERAL_MEMBER)
    verified: bool (Not Null, Default: False)
    primary_shelter_id: int (FK -> Shelter, Nullable)
    profile_photo_url: str (Nullable)
    password_hash: str (Not Null)
```

last_login_at: datetime (Nullable)
failed_login_attempts: **int** (Default: 0)
locked_until: datetime (Nullable)
created_at: datetime (Not Null)
updated_at: datetime (Not Null)
deleted_at: datetime (Nullable) # 軟刪除

關聯關係: - primary_shelter: Many-to-One → Shelter - animals: One-to-Many → Animal
(owner_id) - applications: One-to-Many → Application (applicant_id) - notifications: One-to-Many → Notification (recipient_id)

JSON 輸出格式:

```
{
  "user_id": 1,
  "email": "user@example.com",
  "username": "example_user",
  "phone_number": "0912345678",
  "first_name": "John",
  "last_name": "Doe",
  "region": "台北市",
  "address": {
    "city": "台北市",
    "district": "中正區",
    "street": "重慶南路一段"
  },
  "role": "GENERAL_MEMBER",
  "verified": true,
  "primary_shelter_id": null,
  "profile_photo_url": "https://...",
  "created_at": "2025-12-01T10:00:00+08:00",
  "updated_at": "2025-12-25T12:00:00+08:00"
}
```

2.2 Animal (動物)

檔案位置: backend/app/models/animal.py

資料結構:

```
class Species(Enum):
    CAT = 'CAT'
    DOG = 'DOG'
```

```
class Sex(Enum):
    MALE = 'MALE'
    FEMALE = 'FEMALE'
    UNKNOWN = 'UNKNOWN'
```



```

class AnimalStatus(Enum):
    DRAFT = 'DRAFT'          # 草稿
    SUBMITTED = 'SUBMITTED'  # 已提交審核
    PUBLISHED = 'PUBLISHED'  # 已發布
    ADOPTED = 'ADOPTED'      # 已被領養
    RETIRED = 'RETIRED'      # 已下架

```

```

class Animal(db.Model):
    animal_id: int (PK, Auto)
    name: str (Nullable)
    species: Species (Nullable)
    breed: str (Nullable)      # 品種
    color: str (Nullable)      # 毛色
    sex: Sex (Nullable)
    dob: date (Nullable)       # 出生日期
    description: text (Nullable)
    status: AnimalStatus (Not Null, Default: DRAFT)
    shelter_id: int (FK -> Shelter, Nullable)
    owner_id: int (FK -> User, Nullable)
    medical_summary: text (Nullable)
    rejection_reason: text (Nullable)
    rejected_at: datetime (Nullable)
    rejected_by: int (FK -> User, Nullable)
    created_by: int (FK -> User, Not Null)
    created_at: datetime (Not Null)
    updated_at: datetime (Not Null)
    deleted_at: datetime (Nullable)

```

關聯關係: - shelter: Many-to-One → Shelter - owner: Many-to-One → User - creator: Many-to-One → User - images: One-to-Many → AnimalImage - applications: One-to-Many → Application - medical_records: One-to-Many → MedicalRecord

JSON 輸出格式 (完整版, include_relations=True):

```

{
  "animal_id": 101,
  "name": "小白",
  "species": "DOG",
  "breed": "混種犬",
  "color": "白色",
  "sex": "MALE",
  "dob": "2023-05-15",
  "age": 1,
  "description": "活潑親人的狗狗",
  "status": "PUBLISHED",
  "shelter_id": null,
  "owner_id": 5,

```

```

"medical_summary": "已絕育、已施打疫苗",
"rejection_reason": null,
"rejected_at": null,
"rejected_by": null,
"created_by": 5,
"created_at": "2025-12-01T14:30:00+08:00",
"updated_at": "2025-12-20T09:15:00+08:00",
"images": [
    {
        "animal_image_id": 201,
        "storage_key": "uploads/5/abc123.jpg",
        "url": "http://localhost:9000/petadopt/uploads/5/abc123.jpg",
        "mime_type": "image/jpeg",
        "width": 1920,
        "height": 1080,
        "order": 0
    }
],
"has_pending_application": false
}

```

2.3 Application (領養申請)

檔案位置: backend/app/models/application.py

資料結構:

```

class ApplicationType(Enum):
    ADOPTION = 'ADOPTION'
    REHOME = 'REHOME'

class ApplicationStatus(Enum):
    PENDING = 'PENDING'
    UNDER_REVIEW = 'UNDER_REVIEW'
    APPROVED = 'APPROVED'
    REJECTED = 'REJECTED'
    WITHDRAWN = 'WITHDRAWN'

class Application(db.Model):
    application_id: int (PK, Auto)
    applicant_id: int (FK -> User, Not Null)
    animal_id: int (FK -> Animal, Not Null)
    type: ApplicationType (Not Null)
    status: ApplicationStatus (Not Null, Default: PENDING)
    submitted_at: datetime (Nullable)
    reviewed_at: datetime (Nullable)
    review_notes: text (Nullable)
    assignee_id: int (FK -> User, Nullable)

```

version: `int` (Not Null, Default: 1) # *Optimistic locking*
idempotency_key: `str` (Unique, Nullable)
attachments: JSON (Nullable)
申請人詳細資料
contact_phone: `str` (Nullable)
contact_address: `str` (Nullable)
occupation: `str` (Nullable)
housing_type: `str` (Nullable)
has_experience: `bool` (Nullable)
reason: text (Nullable)
notes: text (Nullable)
created_at: datetime (Not Null)
updated_at: datetime (Not Null)
deleted_at: datetime (Nullable)

JSON 輸出格式:

```
{
  "application_id": 301,
  "applicant_id": 10,
  "animal_id": 101,
  "type": "ADOPTION",
  "status": "PENDING",
  "submitted_at": "2025-12-24T16:30:00+08:00",
  "reviewed_at": null,
  "review_notes": null,
  "assignee_id": null,
  "version": 1,
  "attachments": null,
  "contact_phone": "0912345678",
  "contact_address": "台北市中正區",
  "occupation": "工程師",
  "housing_type": "公寓",
  "has_experience": true,
  "reason": "想給狗狗一個溫暖的家",
  "notes": "平日晚上 7 點後可聯絡",
  "created_at": "2025-12-24T16:30:00+08:00",
  "updated_at": "2025-12-24T16:30:00+08:00"
}
```

2.4 Shelter (收容所)

檔案位置: backend/app/models/shelter.py

資料結構:

```
class Shelter(db.Model):
    shelter_id: int (PK, Auto)
```

name: `str` (Not Null)
slug: `str` (Unique, Nullable)
contact_email: `str` (Not Null)
contact_phone: `str` (Not Null)
address: JSON (Not Null) *# {street, city, county, postal_code}*
region: `str` (Nullable) *# 地區/縣市*
verified: `bool` (Not Null, Default: `False`)
primary_account_user_id: `int` (FK -> User, Nullable)
created_at: datetime (Not Null)
updated_at: datetime (Not Null)
deleted_at: datetime (Nullable)

關聯關係: - primary_account: Many-to-One → User - primary_users: One-to-Many → User (primary_shelter_id) - animals: One-to-Many → Animal

JSON 輸出格式:

```
{
  "shelter_id": 50,
  "name": "台北市動物之家",
  "slug": "taipei-animal-shelter",
  "contact_email": "contact@taipei-shelter.gov.tw",
  "contact_phone": "02-12345678",
  "address": {
    "street": "內湖路一段",
    "city": "台北市",
    "county": "內湖區",
    "postal_code": "114"
  },
  "region": "台北市",
  "verified": true,
  "primary_account_user_id": 8,
  "created_at": "2025-01-15T09:00:00+08:00",
  "updated_at": "2025-12-20T14:30:00+08:00"
}
```

2.5 MedicalRecord (醫療紀錄)

檔案位置: backend/app/models/medical_record.py

資料結構:

```
class RecordType(Enum):
    TREATMENT = 'TREATMENT' # 治療
    CHECKUP = 'CHECKUP' # 健康檢查
    VACCINE = 'VACCINE' # 疫苗接種
    SURGERY = 'SURGERY' # 手術
```

OTHER = 'OTHER' # 其他

```
class MedicalRecord(db.Model):
    medical_record_id: int (PK, Auto)
    animal_id: int (FK -> Animal, Not Null)
    record_type: RecordType (Nullable)
    date: date (Nullable)      # 醫療日期
    provider: str (Nullable)      # 醫療機構/獸醫
    details: text (Nullable)      # 詳細說明
    attachments: JSON (Nullable)      # 附件清單
    verified: bool (Not Null, Default: False)
    verified_by: int (FK -> User, Nullable)
    created_by: int (FK -> User, Nullable)
    created_at: datetime (Not Null)
    updated_at: datetime (Not Null)
    deleted_at: datetime (Nullable)
```

關聯關係: - animal: Many-to-One → Animal - verifier: Many-to-One → User - creator: Many-to-One → User

JSON 輸出格式:

```
{
  "medical_record_id": 401,
  "animal_id": 101,
  "record_type": "VACCINE",
  "date": "2025-12-01",
  "provider": "台北動物醫院",
  "details": "狂犬病疫苗接種 (第二劑)",
  "attachments": [
    {
      "attachment_id": 501,
      "url": "http://localhost:9000/petadopt/uploads/medical/...",
      "filename": "vaccine_certificate.pdf"
    }
  ],
  "verified": true,
  "verified_by": 1,
  "created_by": 5,
  "created_at": "2025-12-01T15:30:00+08:00",
  "updated_at": "2025-12-02T10:00:00+08:00"
}
```

2.6 Notification (通知)

檔案位置: backend/app/models/others.py

資料結構:

```
class Notification(db.Model):
    notification_id: int (PK, Auto)
    recipient_id: int (FK -> User, Not Null)
    actor_id: int (FK -> User, Nullable)
    type: str (Not Null)          # 通知類型
    payload: JSON (Nullable)      # 通知內容
    read: bool (Not Null, Default: False)
    created_at: datetime (Not Null)
    read_at: datetime (Nullable)
```

通知類型: - application_submitted: 申請已提交 - application_approved: 申請已核准 - application_rejected: 申請已拒絕 - animal_published: 動物已發布 - animal_adopted: 動物已領養

JSON 輸出格式:

```
{
  "notification_id": 601,
  "recipient_id": 10,
  "actor_id": 5,
  "type": "application_approved",
  "payload": {
    "application_id": 301,
    "animal_id": 101,
    "animal_name": "小白",
    "message": "您的領養申請已通過審核"
  },
  "read": false,
  "created_at": "2025-12-25T15:00:00+08:00",
  "read_at": null
}
```

2.7 Job (背景任務)

檔案位置: backend/app/models/others.py

資料結構:

```
class JobType(Enum):
    IMPORT_ANIMALS = 'import_animals'
    EXPORT_USER_DATA = 'export_user_data'
    BATCH_NOTIFICATION = 'batch_notification'
    GENERATE_REPORT = 'generate_report'
```

```
class JobStatus(Enum):
    PENDING = 'PENDING'
```

```
RUNNING = 'RUNNING'
SUCCEEDED = 'SUCCEEDED'
FAILED = 'FAILED'
```

```
class Job(db.Model):
    job_id: int (PK, Auto)
    type: str (Not Null)
    status: JobStatus (Not Null, Default: PENDING)
    payload: JSON (Nullable)      # 任務參數
    result_summary: JSON (Nullable) # 執行結果摘要
    created_by: int (FK -> User, Nullable)
    created_at: datetime (Not Null)
    started_at: datetime (Nullable)
    finished_at: datetime (Nullable)
    attempts: int (Not Null, Default: 0)
```

JSON 輸出格式:

```
{
  "job_id": 701,
  "type": "import_animals",
  "status": "SUCCEEDED",
  "payload": {
    "file_url": "uploads/batch/animals_20251225.csv",
    "shelter_id": 50
  },
  "result_summary": {
    "total": 100,
    "success": 95,
    "failed": 5,
    "errors": ["第 10 行: 缺少必填欄位 'name'"]
  },
  "created_by": 8,
  "created_at": "2025-12-25T10:00:00+08:00",
  "started_at": "2025-12-25T10:00:05+08:00",
  "finished_at": "2025-12-25T10:02:30+08:00",
  "attempts": 1
}
```

2.8 Attachment (附件)

檔案位置: backend/app/models/others.py

資料結構:

```
class Attachment(db.Model):
    attachment_id: int (PK, Auto)
    owner_type: str (Not Null)      # 擁有者類型 (e.g., 'medical_record')
```

```

owner_id: int (Not Null)      # 擁有者 ID
storage_key: str (Not Null)   # MinIO 儲存路徑
url: str (Not Null)          # 完整 URL
filename: str (Nullable)     # 原始檔名
mime_type: str (Nullable)    # MIME 類型
size: int (Nullable)         # 檔案大小 (bytes)
meta_data: JSON (Nullable)   # 額外資訊
created_by: int (FK -> User, Nullable)
created_at: datetime (Not Null)
deleted_at: datetime (Nullable)

```

JSON 輸出格式:

```

{
  "attachment_id": 501,
  "owner_type": "medical_record",
  "owner_id": 401,
  "storage_key": "uploads/medical/uuid-abc.pdf",
  "url": "http://localhost:9000/petadopt/uploads/medical/uuid-abc.pdf",
  "filename": "vaccine_certificate.pdf",
  "mime_type": "application/pdf",
  "size": 524288,
  "meta_data": {
    "original_name": "狂犬病疫苗證明.pdf",
    "upload_source": "web"
  },
  "created_by": 5,
  "created_at": "2025-12-01T15:25:00+08:00"
}

```

2.9 AuditLog (稽核日誌)

檔案位置: backend/app/models/others.py

資料結構:

```

class AuditLog(db.Model):
    audit_log_id: int (PK, Auto)
    actor_id: int (FK -> User, Nullable)
    action: str (Not Null)      # 操作類型
    resource_type: str (Not Null) # 資源類型
    resource_id: int (Nullable)  # 資源 ID
    changes: JSON (Nullable)     # 變更內容
    ip_address: str (Nullable)
    user_agent: str (Nullable)
    created_at: datetime (Not Null)

```


JSON 輸出格式:

```
{
  "audit_log_id": 801,
  "actor_id": 1,
  "action": "UPDATE",
  "resource_type": "Application",
  "resource_id": 301,
  "changes": {
    "status": {
      "old": "PENDING",
      "new": "APPROVED"
    },
    "review_notes": {
      "old": null,
      "new": "申請資料完整"
    }
  },
  "ip_address": "192.168.1.100",
  "user_agent": "Mozilla/5.0...",
  "created_at": "2025-12-25T15:00:00+08:00"
}
```

3. REST API 詳細規格

3.1 Authentication APIs

3.1.1 POST /auth/register

描述: 使用者註冊（產生待驗證記錄）

Request Body:

```
{
  "email": "user@example.com",
  "password": "SecurePass123!",
  "username": "example_user",
  "phone_number": "0912345678",
  "region": "台北市",
  "address": {
    "city": "台北市",
    "district": "中正區"
  }
}
```

Response (201 Created):

```
{
  "message": "驗證碼已發送至電子郵件",
}
```

```
"pending_id": 12345,  
"masked_email": "use***@example.com",  
"expires_in": 900  
}
```

錯誤回應 (409 Conflict):

```
{  
  "message": "Email 已被註冊"  
}
```

3.1.2 POST /auth/login

描述: 使用者登入

Request Body:

```
{  
  "email": "user@example.com",  
  "password": "SecurePass123!"  
}
```

Response (200 OK):

```
{  
  "message": "登入成功",  
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",  
  "refresh_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",  
  "user": {  
    "user_id": 1,  
    "email": "user@example.com",  
    "username": "example_user",  
    "role": "GENERAL_MEMBER",  
    "verified": true  
  }  
}
```

錯誤回應 (401 Unauthorized):

```
{  
  "message": "Email 或密碼錯誤"  
}
```

3.2 Animals APIs

3.2.1 GET /animals

描述: 取得動物列表（支援搜尋與篩選）

Query Parameters: - species: 物種 (CAT, DOG) - sex: 性別 (MALE, FEMALE, UNKNOWN) - status: 狀態 (預設: PUBLISHED) - shelter_id: 收容所 ID - source_type: 來源類型 (shelter, personal) - region: 地區/縣市 - min_age: 最小年齡 (月數) - max_age: 最大年齡 (月數) - q: 搜尋關鍵字 - page: 頁碼 (預設: 1) - per_page: 每頁筆數 (預設: 20, 最大: 100)

Request Example:

GET /animals?species=DOG®ion=台北市&page=1&per_page=12

Response (200 OK):

```
{
  "animals": [
    {
      "animal_id": 101,
      "name": "小白",
      "species": "DOG",
      "breed": "混種犬",
      "sex": "MALE",
      "age": 1,
      "status": "PUBLISHED",
      "region": "台北市",
      "images": [
        {
          "animal_image_id": 201,
          "url": "http://localhost:9000/petadopt/uploads/5/abc123.jpg",
          "order": 0
        }
      ],
      "has_pending_application": false
    }
  ],
  "page": 1,
  "per_page": 12,
  "total": 45,
  "pages": 4
}
```

3.2.2 GET /animals/:id

描述: 取得單一動物詳細資訊

Path Parameters: - id: 動物 ID (integer)

Response (200 OK):

```
{
  "animal_id": 101,
  "name": "小白",
  "species": "DOG",
  "breed": "混種犬",
  "color": "白色",
  "sex": "MALE",
  "dob": "2023-05-15",
  "age": 1,
  "description": "活潑親人的狗狗，適合有經驗的飼主。",
  "status": "PUBLISHED",
  "shelter_id": null,
  "owner_id": 5,
  "medical_summary": "已絕育、已施打狂犬病疫苗",
  "created_by": 5,
  "created_at": "2025-12-01T14:30:00+08:00",
  "updated_at": "2025-12-20T09:15:00+08:00",
  "images": [
    {
      "animal_image_id": 201,
      "storage_key": "uploads/5/abc123.jpg",
      "url": "http://localhost:9000/petadopt/uploads/5/abc123.jpg",
      "mime_type": "image/jpeg",
      "width": 1920,
      "height": 1080,
      "order": 0
    }
  ],
  "owner": {
    "user_id": 5,
    "username": "john_doe",
    "region": "台北市"
  },
  "has_pending_application": false
}
```

錯誤回應 (404 Not Found):

```
{
  "message": "動物不存在"
}
```

3.2.3 POST /animals

描述: 建立動物資料 (送養刊登) **需要認證:** Yes (JWT Bearer Token)

Request Headers:

Authorization: Bearer <access_token>

Content-Type: application/json

Request Body:

```
{
  "name": "小花",
  "species": "CAT",
  "breed": "米克斯",
  "color": "橘色",
  "sex": "FEMALE",
  "dob": "2024-03-10",
  "description": "溫柔親人的貓咪",
  "medical_summary": "已絕育",
  "status": "DRAFT"
}
```

Response (201 Created):

```
{
  "message": "動物資料建立成功",
  "animal": {
    "animal_id": 102,
    "name": "小花",
    "species": "CAT",
    "status": "DRAFT",
    "created_by": 5,
    "created_at": "2025-12-25T12:30:00+08:00"
  }
}
```

3.2.4 PATCH /animals/:id

描述: 更新動物資訊 **需要認證:** Yes **權限:** 僅限動物擁有者或管理員

Request Body (部分欄位):

```
{
  "description": "更新後的描述",
  "medical_summary": "已完成三合一疫苗"
}
```

Response (200 OK):

```
{
  "message": "動物資料更新成功",
  "animal": { /* 完整 animal 物件 */ }
}
```

3.3 Applications APIs

3.3.1 GET /applications

描述: 取得申請列表 **需要認證:** Yes

Query Parameters: - mode: 查詢模式 - all: 所有相關申請 (管理員) - my: 我提交的申請 (一般會員) - review: 待我審核的申請 (收容所/管理員) - status: 申請狀態篩選 - animal_id: 特定動物的申請 - page, per_page: 分頁參數

Response (200 OK):

```
{
  "items": [
    {
      "application_id": 301,
      "applicant_id": 10,
      "animal_id": 101,
      "type": "ADOPTION",
      "status": "PENDING",
      "submitted_at": "2025-12-24T16:30:00+08:00",
      "contact_phone": "0912345678",
      "reason": "想給狗狗一個溫暖的家",
      "applicant": {
        "user_id": 10,
        "username": "adopter_01"
      },
      "animal": {
        "animal_id": 101,
        "name": "小白",
        "species": "DOG"
      }
    }
  ],
  "total": 15,
  "page": 1,
  "per_page": 20,
  "pages": 1
}
```

3.3.2 POST /applications

描述: 建立領養申請 **需要認證:** Yes (GENERAL_MEMBER only)

Request Headers:

Authorization: Bearer <access_token>
Idempotency-Key: <unique_key> // 可選，用於防止重複提交
Content-Type: application/json

Request Body:

```
{  
  "animal_id": 101,  
  "type": "ADOPTION",  
  "contact_phone": "0912345678",  
  "contact_address": "台北市中正區重慶南路",  
  "occupation": "軟體工程師",  
  "housing_type": "公寓",  
  "has_experience": true,  
  "reason": "我很喜歡狗狗，有充足時間陪伴",  
  "notes": "平日晚上 7 點後可聯絡"  
}
```

Response (201 Created):

```
{  
  "message": "申請已提交",  
  "application": {  
    "application_id": 302,  
    "applicant_id": 10,  
    "animal_id": 101,  
    "status": "PENDING",  
    "submitted_at": "2025-12-25T14:00:00+08:00"  
  }  
}
```

Business Rules: 1. 申請者不能對自己刊登的動物提出申請 2. 同一動物只能提交一次申請（透過 idempotency_key 確保） 3. 僅限 GENERAL_MEMBER 角色

3.3.3 POST /applications/:id/review

描述: 審核申請（核准/拒絕） **需要認證:** Yes **權限:** 動物擁有者、收容所管理員、系統管理員

Request Body:

```
{  
  "action": "approve",  
  "review_notes": "申請資料完整，條件符合",  
  "version": 1  
}
```

Response (200 OK):

```
{
  "message": "申請已核准",
  "application": {
    "application_id": 301,
    "status": "APPROVED",
    "reviewed_at": "2025-12-25T15:00:00+08:00",
    "review_notes": "申請資料完整，條件符合"
  }
}
```

3.4 Uploads APIs

3.4.1 POST /uploads/direct

描述: 直接上傳檔案（後端代理至 MinIO） **需要認證:** Yes

Request: - Content-Type: multipart/form-data - Body: file (binary)

Response (200 OK):

```
{
  "message": "檔案上傳成功",
  "storage_key": "uploads/5/uuid-123.jpg",
  "url": "http://localhost:9000/petadopt/uploads/5/uuid-123.jpg",
  "size": 2048576,
  "mime_type": "image/jpeg"
}
```

3.4.2 POST /uploads/presign

描述: 取得預簽名 URL（前端直接上傳至 MinIO） **需要認證:** Yes

Request Body:

```
{
  "filename": "my-dog-photo.jpg",
  "mime_type": "image/jpeg",
  "size": 2048576
}
```

Response (200 OK):

```
{
  "presigned_url": "http://localhost:9000/petadopt/uploads/...",
  "storage_key": "uploads/5/uuid-456.jpg",
  "expires_in": 3600
}
```

3.5 Medical Records APIs

3.5.1 GET /medical-records/animals

描述: 取得當前使用者有權限管理醫療紀錄的動物列表 **需要認證:** Yes

Query Parameters: - name: 動物名稱搜尋 - species: 物種篩選 - breed: 品種篩選 - min_age, max_age: 年齡範圍 - adopted: 是否已領養

Response (200 OK):

```
{
  "animals": [
    {
      "animal_id": 101,
      "name": "小白",
      "species": "DOG",
      "breed": "混種犬",
      "age": 1,
      "status": "PUBLISHED"
    }
  ],
  "total": 5
}
```

3.5.2 POST /medical-records/animals/:animal_id/medical-records

描述: 為動物建立醫療紀錄 **需要認證:** Yes **權限:** 動物擁有者、收容所成員、管理員

Request Body:

```
{
  "record_type": "VACCINE",
  "date": "2025-12-25",
  "provider": "台北動物醫院",
  "details": "狂犬病疫苗接種",
  "attachments": [
    {
      "storage_key": "uploads/medical/uuid-123.pdf",
      "url": "http://...",
      "filename": "vaccine_cert.pdf",
      "mime_type": "application/pdf"
    }
  ]
}
```

Response (201 Created):

```
{
  "message": "醫療紀錄創建成功",
  "medical_record": {
    "medical_record_id": 401,
    "animal_id": 101,
    "record_type": "VACCINE",
    "date": "2025-12-25",
    "provider": "台北動物醫院",
    "details": "狂犬病疫苗接種",
    "verified": false
  }
}
```

3.5.3 GET /medical-records/animals/:animal_id/medical-records

描述: 取得動物的醫療紀錄列表

Response (200 OK):

```
{
  "medical_records": [
    {
      "medical_record_id": 401,
      "record_type": "VACCINE",
      "date": "2025-12-25",
      "provider": "台北動物醫院",
      "details": "狂犬病疫苗接種",
      "verified": true,
      "attachments": [...]
    }
  ],
  "total": 3
}
```

3.5.4 PATCH /medical-records/:id

描述: 更新醫療紀錄 **需要認證:** Yes **權限:** 建立者或管理員

Request Body (部分欄位):

```
{
  "details": "更新後的詳細說明",
  "verified": true
}
```

Response (200 OK):

```
{
  "message": "醫療紀錄更新成功",
  "medical_record": { /* 完整醫療紀錄 */ }
}
```

3.6 Shelters APIs

3.6.1 GET /shelters

描述: 取得收容所列表 (公開端點)

Query Parameters: - page: 頁碼 - per_page: 每頁筆數 - search: 搜尋關鍵字 - verified: 是否只顯示已驗證的收容所 (true/false)

Response (200 OK):

```
{
  "shelters": [
    {
      "shelter_id": 50,
      "name": "台北市動物之家",
      "slug": "taipei-animal-shelter",
      "region": "台北市",
      "verified": true,
      "contact_phone": "02-12345678"
    }
  ],
  "page": 1,
  "per_page": 10,
  "total": 25,
  "pages": 3
}
```

3.6.2 POST /shelters

描述: 建立收容所 **需要認證:** Yes **權限:** SHELTER_MEMBER 或 ADMIN

Request Body:

```
{
  "name": "新竹市動物收容所",
  "contact_email": "contact@hsinchu-shelter.gov.tw",
  "contact_phone": "03-12345678",
  "address": {
    "street": "東大路一段",
    "city": "新竹市",
    "county": "東區",
  }
}
```

```
    "postal_code": "300"
  },
  "region": "新竹市"
}
```

Response (201 Created):

```
{
  "message": "收容所創建成功",
  "shelter": {
    "shelter_id": 51,
    "name": "新竹市動物收容所",
    "verified": false,
    "primary_account_user_id": 8
  }
}
```

3.6.3 GET /shelters/:id

描述: 取得收容所詳細資訊（公開端點）

Response (200 OK):

```
{
  "shelter_id": 50,
  "name": "台北市動物之家",
  "slug": "taipei-animal-shelter",
  "contact_email": "contact@taipei-shelter.gov.tw",
  "contact_phone": "02-12345678",
  "address": {
    "street": "內湖路一段",
    "city": "台北市",
    "county": "內湖區",
    "postal_code": "114"
  },
  "region": "台北市",
  "verified": true,
  "primary_account_user_id": 8,
  "statistics": {
    "total_animals": 150,
    "published_animals": 120,
    "adopted_animals": 30
  }
}
```

3.6.4 POST /shelters/:id/animals/batch

描述: 批次匯入動物資料 (CSV/JSON) **需要認證:** Yes **權限:** 收容所管理員

Request Body (JSON 格式):

```
{
  "animals": [
    {
      "name": "小黑",
      "species": "DOG",
      "breed": "拉布拉多",
      "sex": "MALE",
      "dob": "2023-01-15",
      "description": "友善活潑"
    },
    {
      "name": "小花",
      "species": "CAT",
      "breed": "米克斯",
      "sex": "FEMALE",
      "dob": "2024-03-10"
    }
  ]
}
```

Response (202 Accepted):

```
{
  "message": "批次匯入任務已建立",
  "job_id": 701,
  "status": "PENDING",
  "estimated_duration": 120
}
```

後續查詢任務狀態: GET /jobs/:job_id

3.7 Notifications APIs

3.7.1 GET /notifications

描述: 取得當前使用者的通知列表 **需要認證:** Yes

Query Parameters: - unread_only: 只顯示未讀通知 (true/false) - type: 通知類型篩選 - page, per_page: 分頁參數

Response (200 OK):

```
{
  "notifications": [
    {
      "notification_id": 601,
      "type": "application_approved",
      "payload": {
        "application_id": 301,
        "animal_name": "小白",
        "message": "您的領養申請已通過審核"
      },
      "read": false,
      "created_at": "2025-12-25T15:00:00+08:00",
      "actor": {
        "user_id": 5,
        "username": "john_doe"
      }
    }
  ],
  "total": 15,
  "unread_count": 3,
  "page": 1,
  "per_page": 20
}
```

3.7.2 POST /notifications/:id/read

描述: 標記通知為已讀 **需要認證:** Yes

Response (200 OK):

```
{
  "message": "通知已標記為已讀",
  "notification": {
    "notification_id": 601,
    "read": true,
    "read_at": "2025-12-25T16:00:00+08:00"
  }
}
```

3.7.3 POST /notifications/read-all

描述: 標記所有通知為已讀 **需要認證:** Yes

Response (200 OK):

```
{
  "message": "所有通知已標記為已讀",
}
```

```
"updated_count": 5
}
```

3.7.4 GET /notifications/unread-count

描述: 取得未讀通知數量 **需要認證:** Yes

Response (200 OK):

```
{
  "unread_count": 3
}
```

3.8 Jobs APIs

3.8.1 GET /jobs/:id

描述: 取得任務狀態 **需要認證:** Yes

Response (200 OK):

```
{
  "job_id": 701,
  "type": "import_animals",
  "status": "RUNNING",
  "payload": {
    "shelter_id": 50,
    "file_url": "uploads/batch/animals.csv"
  },
  "result_summary": null,
  "created_at": "2025-12-25T10:00:00+08:00",
  "started_at": "2025-12-25T10:00:05+08:00",
  "finished_at": null,
  "attempts": 1,
  "progress": {
    "current": 50,
    "total": 100,
    "percentage": 50
  }
}
```

3.8.2 GET /jobs

描述: 取得任務列表 **需要認證:** Yes

Query Parameters: - status: 任務狀態篩選 - type: 任務類型篩選 - page, per_page: 分頁參數

Response (200 OK):

```
{
  "jobs": [
    {
      "job_id": 701,
      "type": "import_animals",
      "status": "SUCCEEDED",
      "created_at": "2025-12-25T10:00:00+08:00",
      "finished_at": "2025-12-25T10:02:30+08:00"
    }
  ],
  "total": 10,
  "page": 1,
  "per_page": 20
}
```

3.9 Users APIs

3.9.1 GET /users/:id

描述: 取得使用者公開資料

Response (200 OK):

```
{
  "user_id": 5,
  "username": "john_doe",
  "region": "台北市",
  "profile_photo_url": "http://...",
  "verified": true,
  "created_at": "2025-01-01T00:00:00+08:00"
}
```

3.9.2 PATCH /users/:id

描述: 更新使用者資料 **需要認證:** Yes **權限:** 本人或管理員

Request Body:

```
{
  "username": "new_username",
  "phone_number": "0987654321",
  "region": "新北市",
  "address": {
```



```
"city": "新北市",
"district": "板橋區"
}
}
```

Response (200 OK):

```
{
  "message": "使用者資料更新成功",
  "user": { /* 完整使用者資料 */ }
}
```

3.10 Admin APIs

3.10.1 GET /admin/users

描述: 取得所有使用者列表（管理後台） **需要認證:** Yes **權限:** ADMIN only

Query Parameters: - role: 角色篩選 - verified: 驗證狀態篩選 - search: 搜尋 email/username - page, per_page: 分頁參數

Response (200 OK):

```
{
  "users": [
    {
      "user_id": 1,
      "email": "admin@example.com",
      "username": "admin",
      "role": "ADMIN",
      "verified": true,
      "last_login_at": "2025-12-25T09:00:00+08:00"
    }
  ],
  "total": 500,
  "page": 1,
  "per_page": 50
}
```

3.10.2 POST /admin/users/:id/update-role

描述: 更新使用者角色 **需要認證:** Yes **權限:** ADMIN only

Request Body:

```
{
  "role": "SHELTER_MEMBER"
}
```

Response (200 OK):

```
{
  "message": "使用者角色已更新",
  "user": {
    "user_id": 10,
    "role": "SHELTER_MEMBER"
  }
}
```

3.10.3 GET /admin/audit-logs

描述: 取得稽核日誌 **需要認證:** Yes **權限:** ADMIN only

Query Parameters: - actor_id: 操作者 ID - action: 操作類型 (CREATE, UPDATE, DELETE) - resource_type: 資源類型 - start_date, end_date: 日期範圍 - page, per_page: 分頁參數

Response (200 OK):

```
{
  "logs": [
    {
      "audit_log_id": 801,
      "actor_id": 1,
      "action": "UPDATE",
      "resource_type": "Application",
      "resource_id": 301,
      "changes": {
        "status": {"old": "PENDING", "new": "APPROVED"}
      },
      "ip_address": "192.168.1.100",
      "created_at": "2025-12-25T15:00:00+08:00"
    }
  ],
  "total": 1000,
  "page": 1,
  "per_page": 50
}
```

4. 前端資料結構 (TypeScript Interfaces)

4.1 Animals API Types

檔案位置: frontend/src/api/animals.ts

```

export interface AnimalFilters {
  species?: 'CAT' | 'DOG'
  breed?: string
  sex?: 'MALE' | 'FEMALE' | 'UNKNOWN'
  status?: 'DRAFT' | 'SUBMITTED' | 'PUBLISHED' | 'ADOPTED' | 'RETIRED'
  shelter_id?: number
  owner_id?: number
  source_type?: 'shelter' | 'personal'
  region?: string
  min_age?: number
  max_age?: number
  featured?: boolean
  q?: string
  page?: number
  per_page?: number
}

```

```

export interface Animal {
  animal_id: number
  name?: string
  species?: 'CAT' | 'DOG'
  breed?: string
  color?: string
  sex?: 'MALE' | 'FEMALE' | 'UNKNOWN'
  dob?: string
  age?: number
  description?: string
  status: 'DRAFT' | 'SUBMITTED' | 'PUBLISHED' | 'ADOPTED' | 'RETIRED'
  shelter_id?: number
  owner_id?: number
  medical_summary?: string
  rejection_reason?: string
  rejected_at?: string
  rejected_by?: number
  created_by: number
  created_at: string
  updated_at: string
  deleted_at?: string
  images?: Array<{
    animal_image_id: number
    storage_key: string
    url: string
    mime_type?: string
    width?: number
    height?: number
    order: number
  }>
  featured?: boolean
  has_pending_application?: boolean
}

```

```
export interface AnimalsResponse {  
  animals: Animal[]  
  page: number  
  per_page: number  
  total: number  
  pages: number  
}
```

4.2 Applications API Types

檔案位置: frontend/src/api/applications.ts

```
export interface Application {  
  application_id: number  
  applicant_id: number  
  animal_id: number  
  type: 'ADOPTION' | 'REHOME'  
  status: 'PENDING' | 'UNDER_REVIEW' | 'APPROVED' | 'REJECTED' | 'WITHDRAWN'  
  submitted_at?: string  
  reviewed_at?: string  
  review_notes?: string  
  assignee_id?: number  
  version: number  
  attachments?: any  
  contact_phone?: string  
  contact_address?: string  
  occupation?: string  
  housing_type?: string  
  has_experience?: boolean  
  reason?: string  
  notes?: string  
  created_at: string  
  updated_at: string  
  applicant?: any  
  animal?: any  
  assignee?: any  
}
```

```
export interface CreateApplicationData {  
  animal_id: number  
  type?: 'ADOPTION' | 'REHOME'  
  attachments?: any  
  contact_phone?: string  
  contact_address?: string  
  occupation?: string  
  housing_type?: string  
  has_experience?: boolean
```

```
reason?: string
notes?: string
}
```

```
export interface ReviewApplicationData {
  action: 'approve' | 'reject'
  review_notes?: string
  version?: number
}
```

4.3 Medical Records API Types

檔案位置: frontend/src/api/medicalRecords.ts

```
export interface MedicalRecord {
  medical_record_id: number
  animal_id: number
  record_type?: 'TREATMENT' | 'CHECKUP' | 'VACCINE' | 'SURGERY' | 'OTHER'
  date?: string
  provider?: string
  details?: string
  attachments?: Array<{
    attachment_id: number
    url: string
    filename?: string
    mime_type?: string
  }>
  verified: boolean
  verified_by?: number
  created_by?: number
  created_at: string
  updated_at: string
}
```

```
export interface CreateMedicalRecordData {
  animal_id: number
  record_type?: 'TREATMENT' | 'CHECKUP' | 'VACCINE' | 'SURGERY' | 'OTHER'
  date?: string
  provider?: string
  details?: string
  attachments?: Array<{
    storage_key: string
    url: string
    filename?: string
    mime_type?: string
  }>
}
```

4.4 Shelters API Types

檔案位置: frontend/src/api/shelters.ts

```
export interface Shelter {  
  shelter_id: number  
  name: string  
  slug?: string  
  contact_email: string  
  contact_phone: string  
  address: {  
    street: string  
    city: string  
    county: string  
    postal_code: string  
  }  
  region?: string  
  verified: boolean  
  primary_account_user_id?: number  
  created_at: string  
  updated_at: string  
}
```

```
export interface ShelterFilters {  
  page?: number  
  per_page?: number  
  search?: string  
  verified?: boolean  
}
```

```
export interface CreateShelterData {  
  name: string  
  contact_email: string  
  contact_phone: string  
  address: {  
    street: string  
    city: string  
    county: string  
    postal_code: string  
  }  
  region?: string  
}
```

4.5 Notifications API Types

檔案位置: frontend/src/api/notifications.ts

```

export interface Notification {
  notification_id: number
  recipient_id: number
  actor_id?: number
  type: string
  payload: {
    [key: string]: any
  }
  read: boolean
  created_at: string
  read_at?: string
  actor?: {
    user_id: number
    username?: string
  }
}

export interface NotificationsResponse {
  notifications: Notification[]
  total: number
  unread_count: number
  page: number
  per_page: number
}

export interface NotificationFilters {
  unread_only?: boolean
  type?: string
  page?: number
  per_page?: number
}

```

4.6 Jobs API Types

檔案位置: frontend/src/api/jobs.ts

```

export interface Job {
  job_id: number
  type: string
  status: 'PENDING' | 'RUNNING' | 'SUCCEEDED' | 'FAILED'
  payload?: {
    [key: string]: any
  }
  result_summary?: {
    total?: number
    success?: number
    failed?: number
    errors?: string[]
  }
}

```

```

    }
    created_by?: number
    created_at: string
    started_at?: string
    finished_at?: string
    attempts: number
    progress?: {
      current: number
      total: number
      percentage: number
    }
  }
}

export interface JobsResponse {
  jobs: Job[]
  total: number
  page: number
  per_page: number
}

```

4.7 Uploads API Types

檔案位置: frontend/src/api/uploads.ts

```

export interface UploadDirectResponse {
  message: string
  storage_key: string
  url: string
  size: number
  mime_type: string
}

```

```

export interface PresignResponse {
  presigned_url: string
  storage_key: string
  expires_in: number
}

```

```

export interface PresignRequest {
  filename: string
  mime_type: string
  size: number
}

```

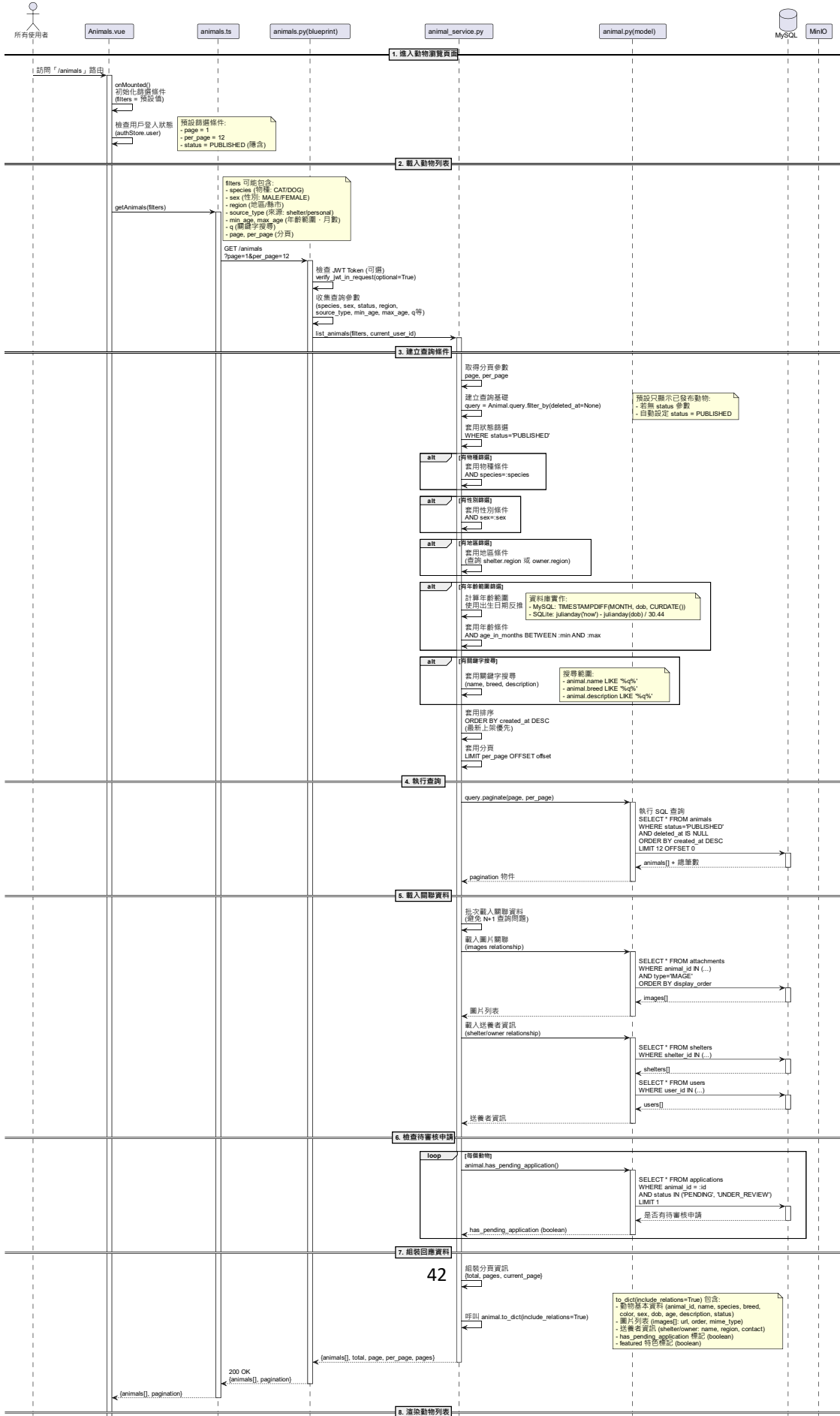
5. 循序圖索引

以下為關鍵 Use Case 的循序圖：

5.1 動物瀏覽與搜尋

1_1: 動物列表瀏覽

使用案例1.1: 動物列表瀏覽



動物列表瀏覽

1_2: 動物搜尋與篩選

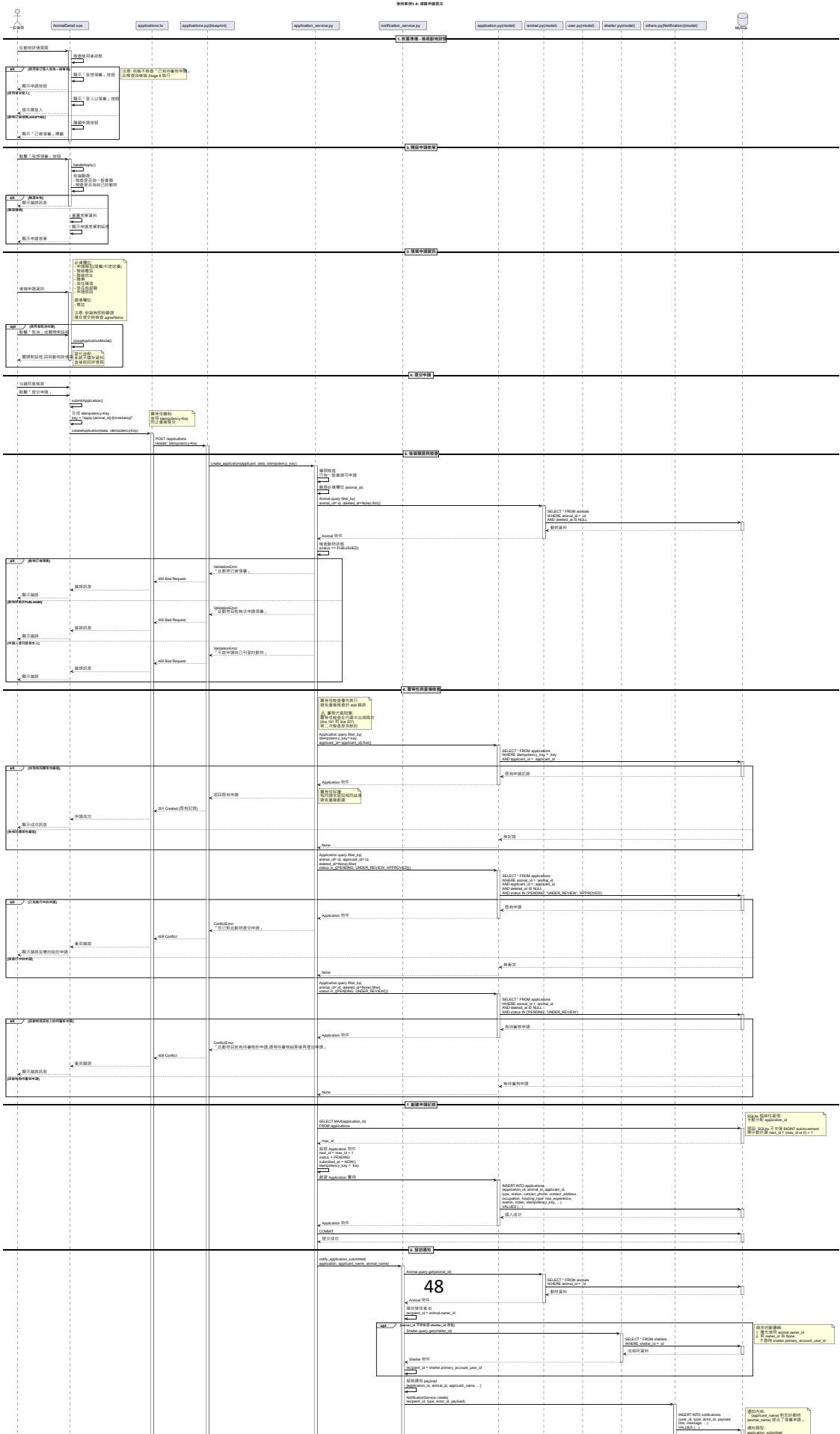
動物搜尋與篩選

1_3: 動物詳情檢視



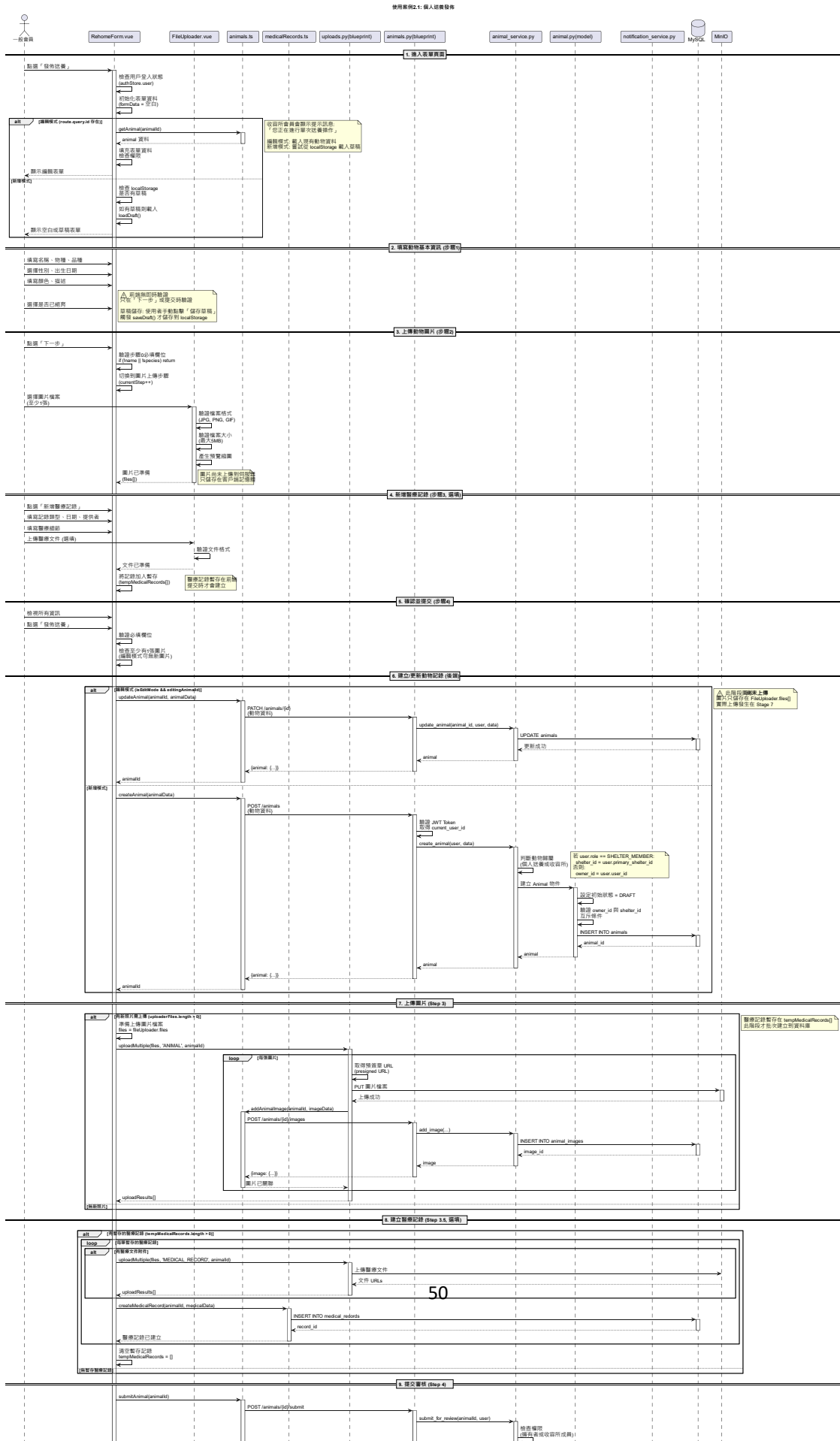
動物詳情檢視

1_4: 動物圖片檢視



5.2 個人送養流程

2_1: 個人送養發佈

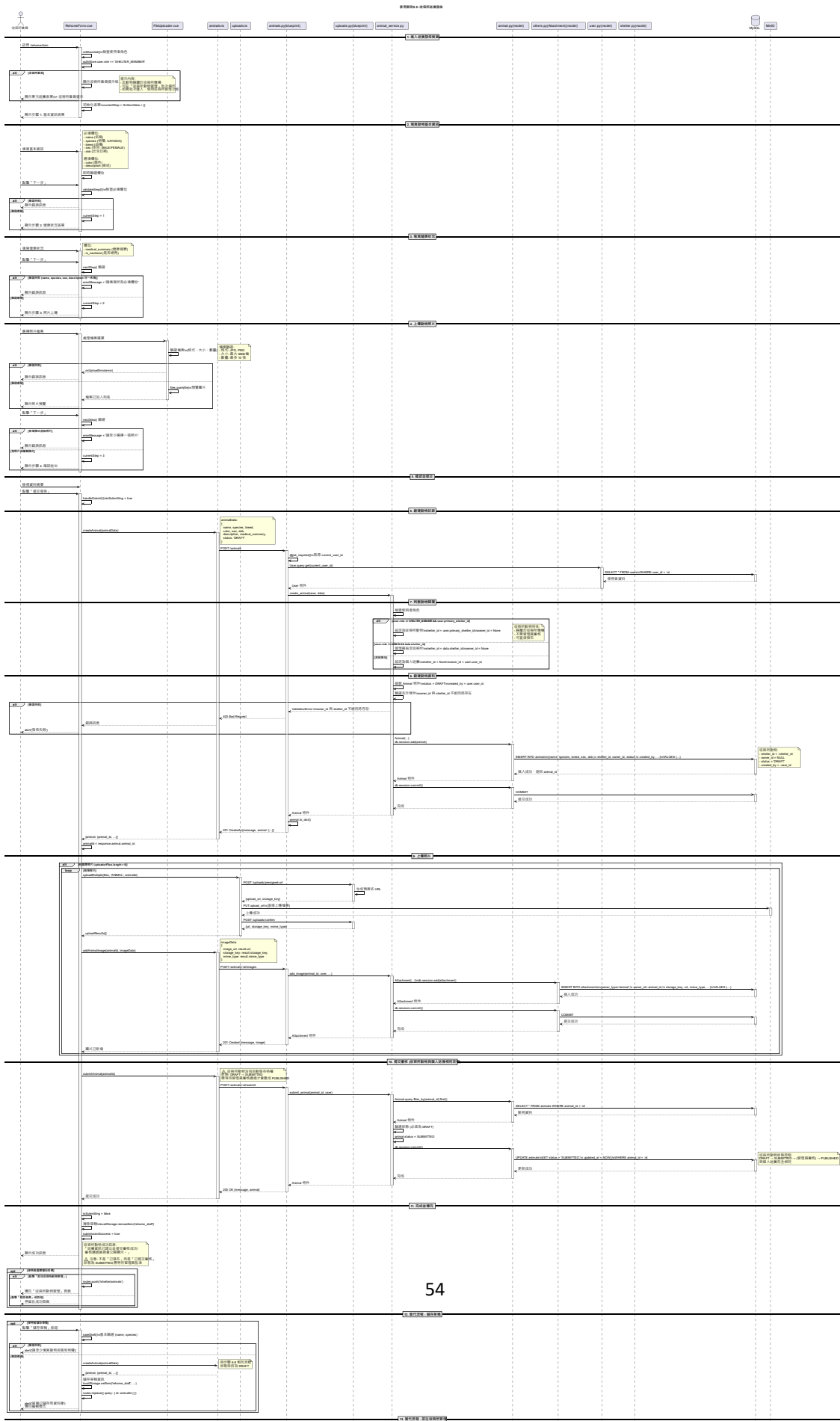


個人送養發佈

2_2: 送養草稿儲存

送養草稿儲存

2_3: 送養提交審核

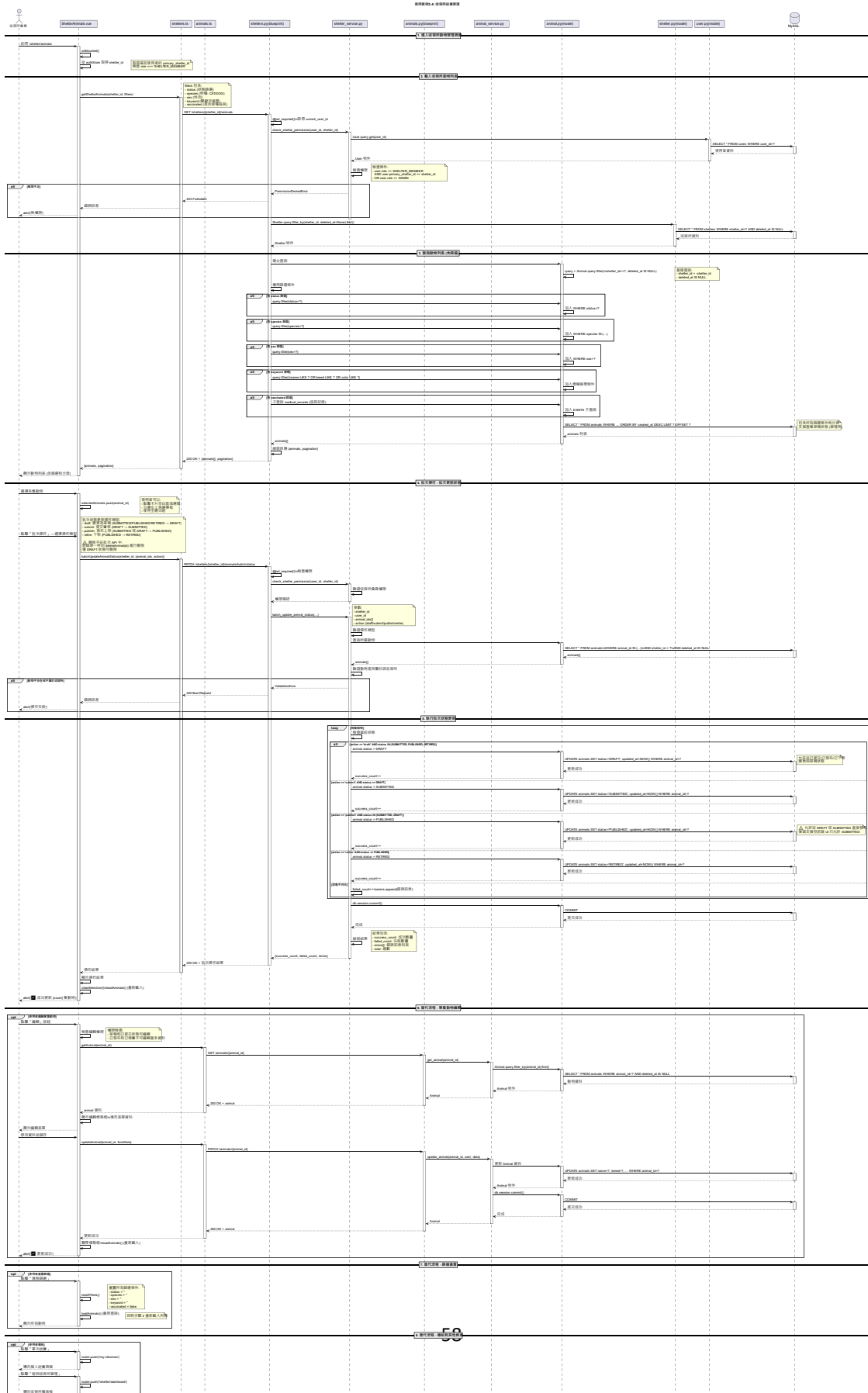


送養提交審核

2_3_1: 送養提交審核（子流程）

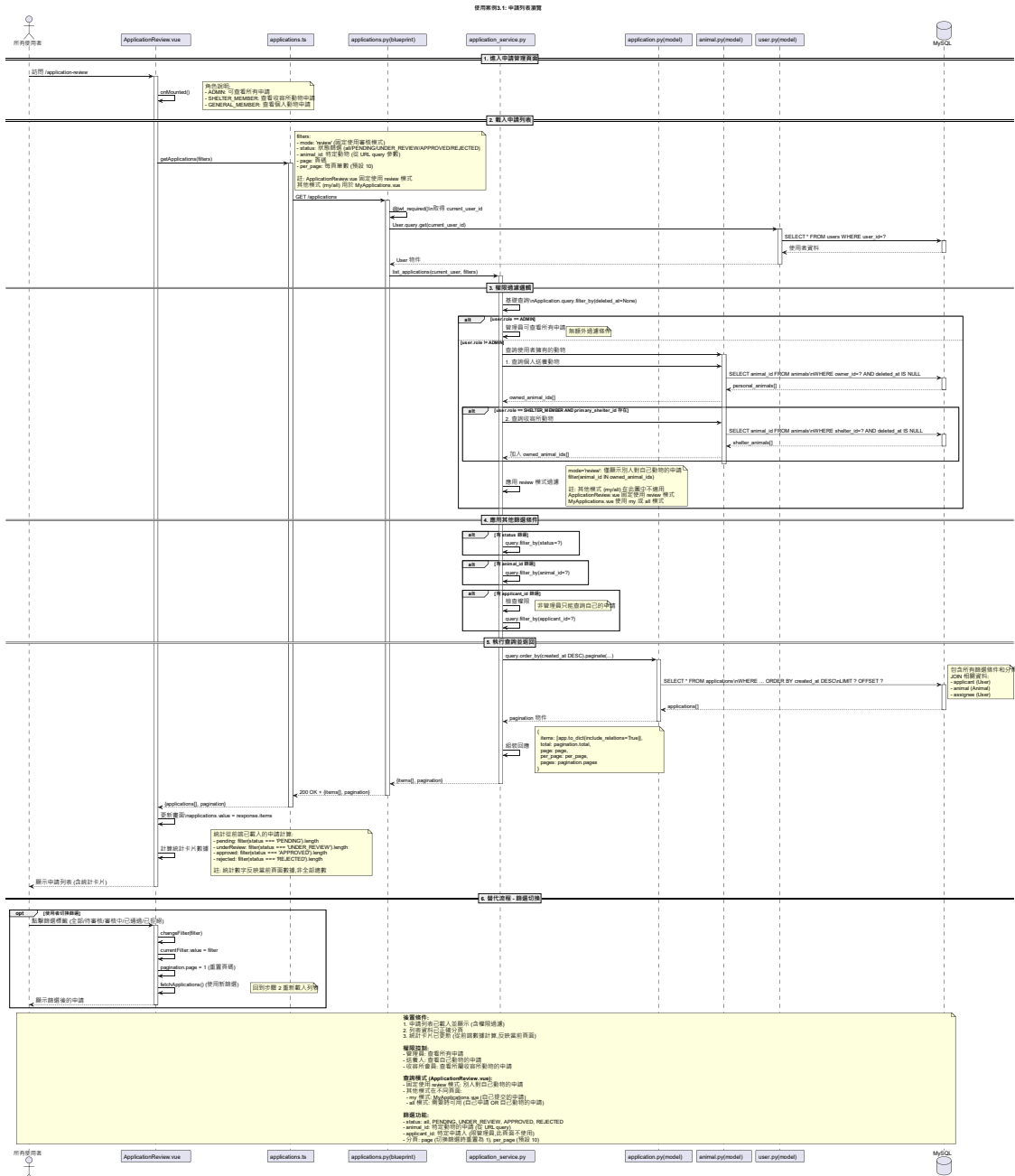
送養提交審核子流程

2_4: 送養編輯與下架



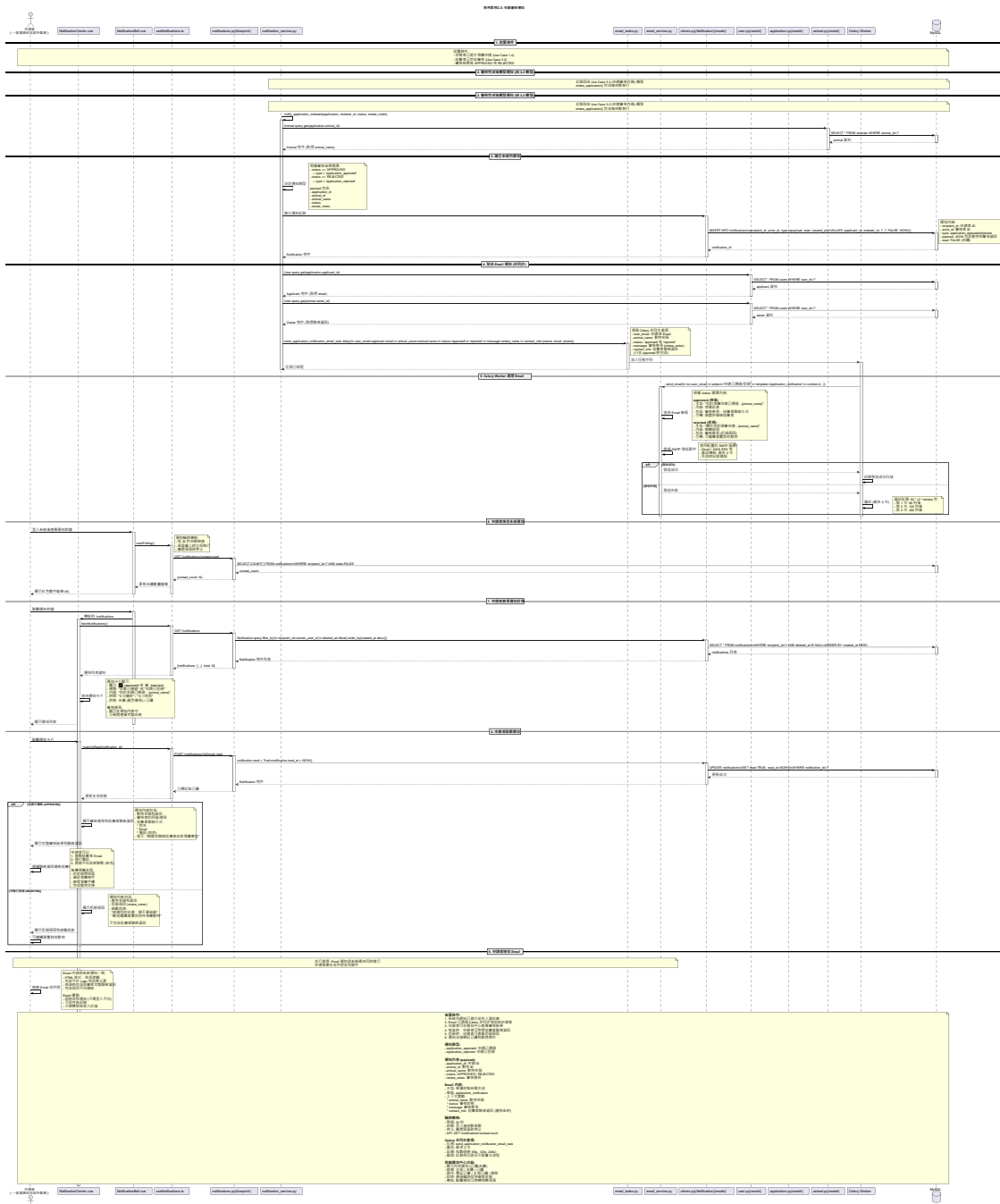
5.3 領養申請流程

3_1: 提出領養申請



提出領養申請

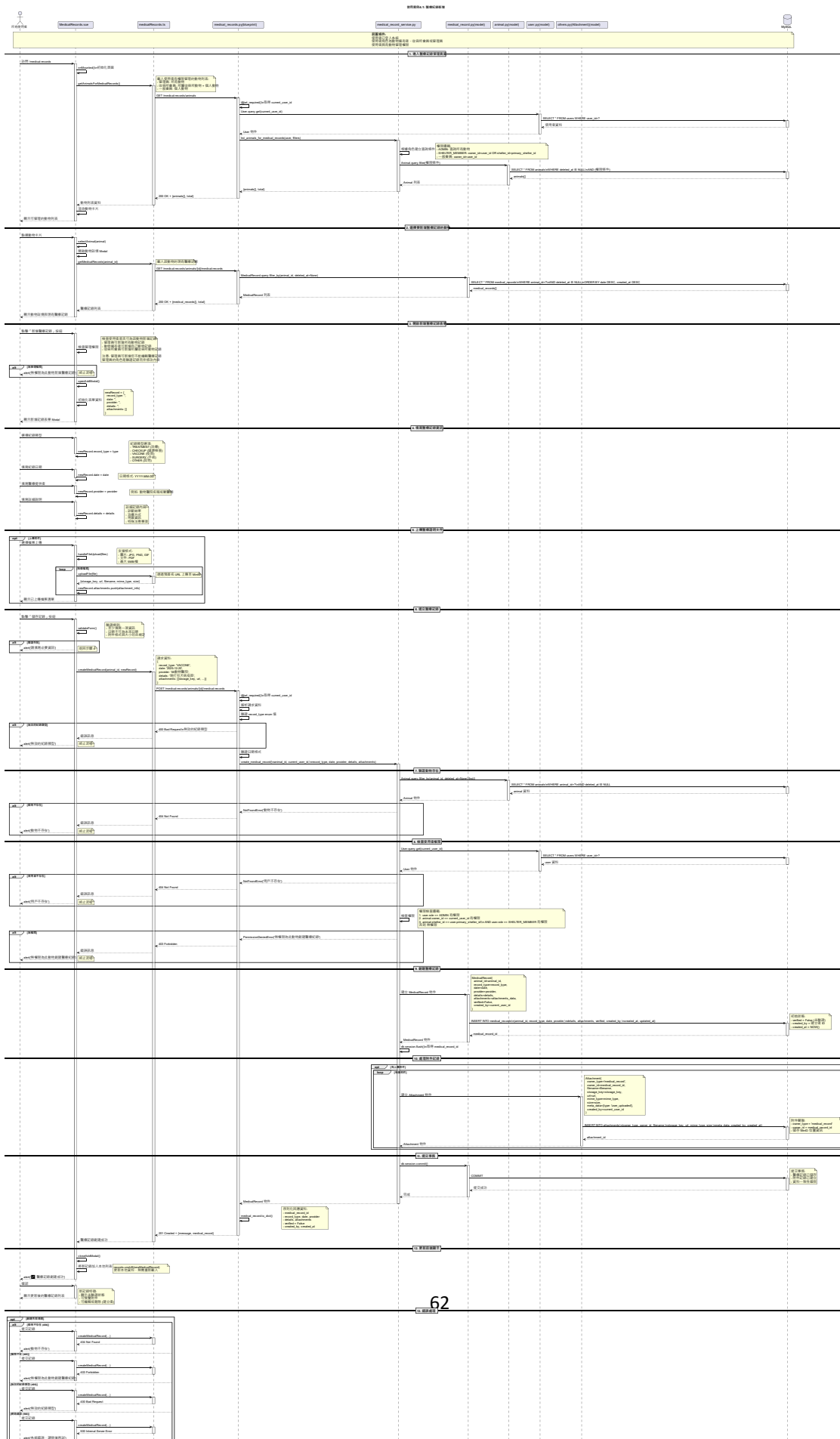
3_2: 查看申請狀態



撤回申請

5.4 審核流程

4_1: 管理員審核刊登



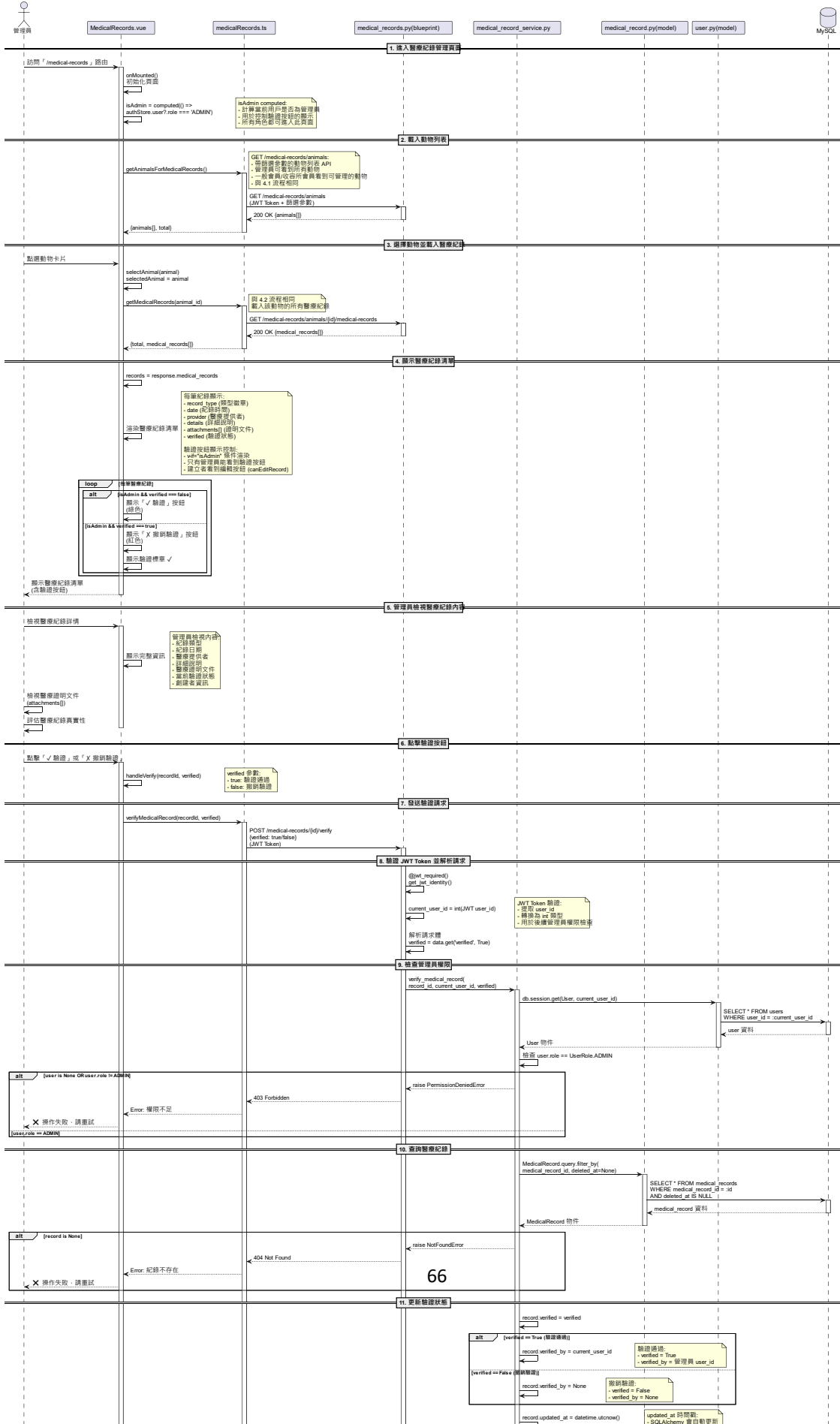
管理員審核刊登

4_2: 審核領養申請

審核領養申請

4_3: 拒絕與退回

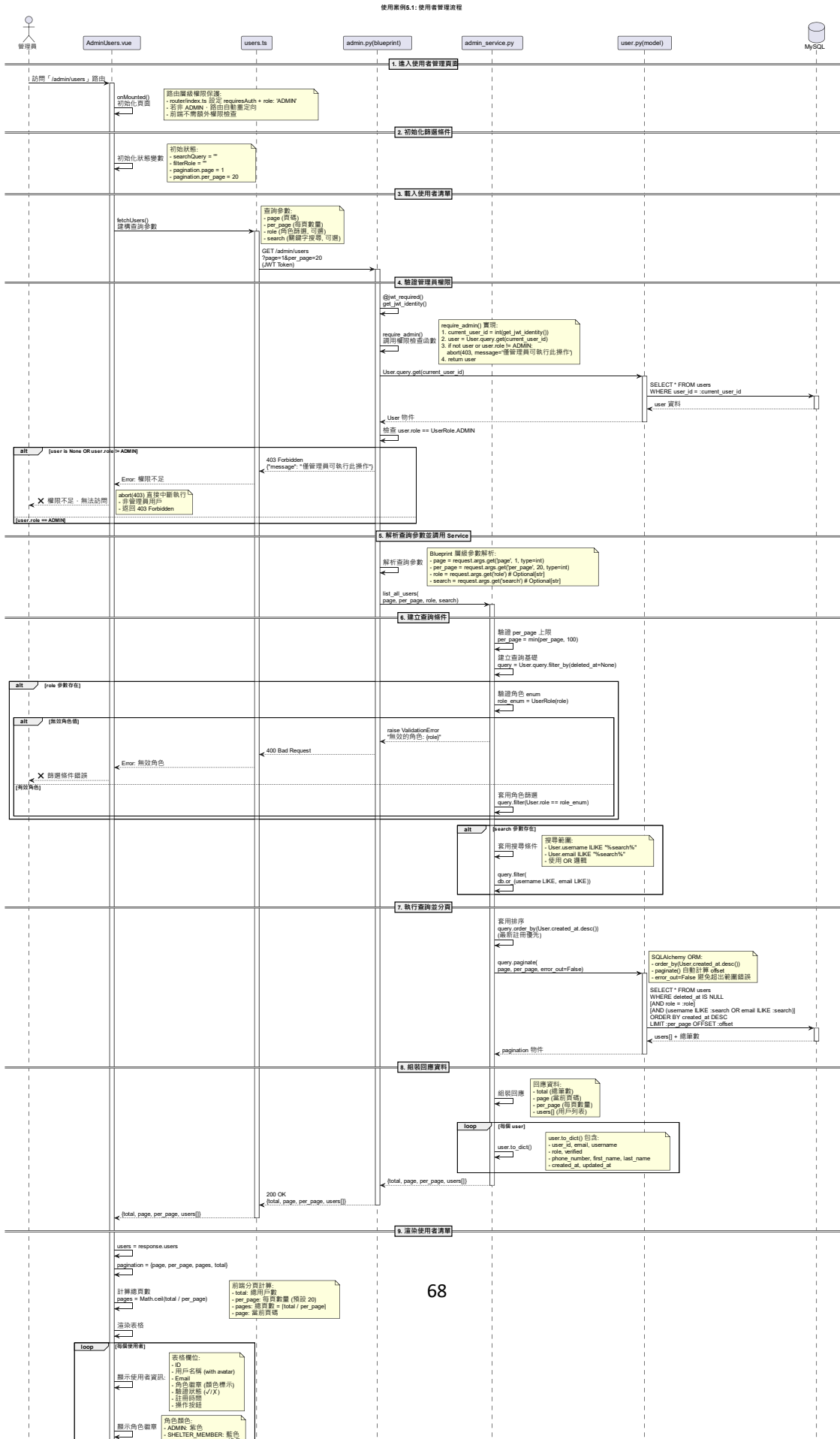
使用案例4.3: 醫療紀錄驗證流程



拒絕與退回

5.5 收容所管理

5_1: 收容所批次上傳

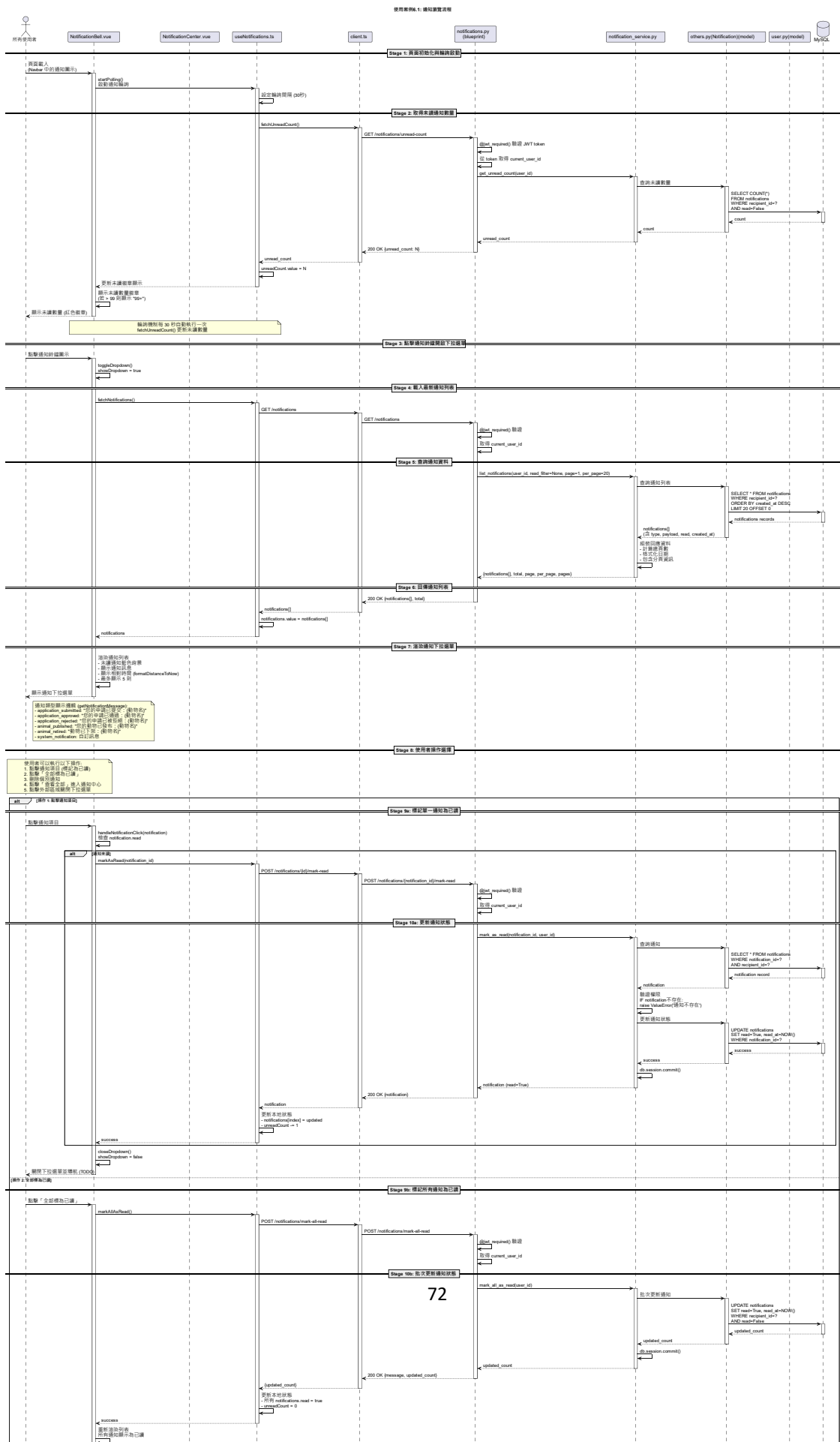


收容所批次上傳

5_2: 收容所動物管理

5.6 醫療紀錄

6_1: 新增醫療紀錄



新增醫療紀錄

6_2: 查看醫療歷史

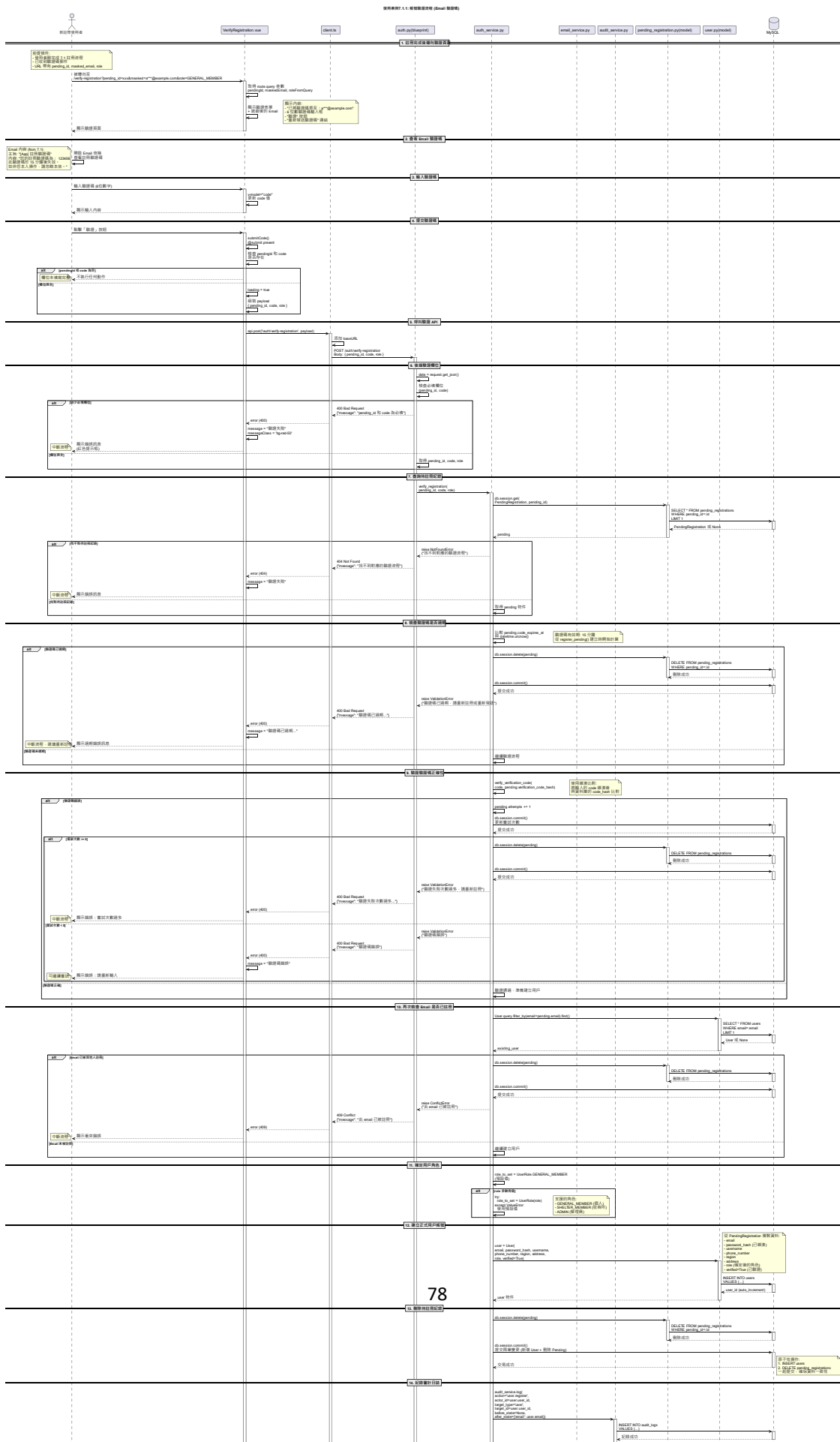
查看醫療歷史

5.7 通知系統

7_1: 通知產生與發送

通知產生與發送

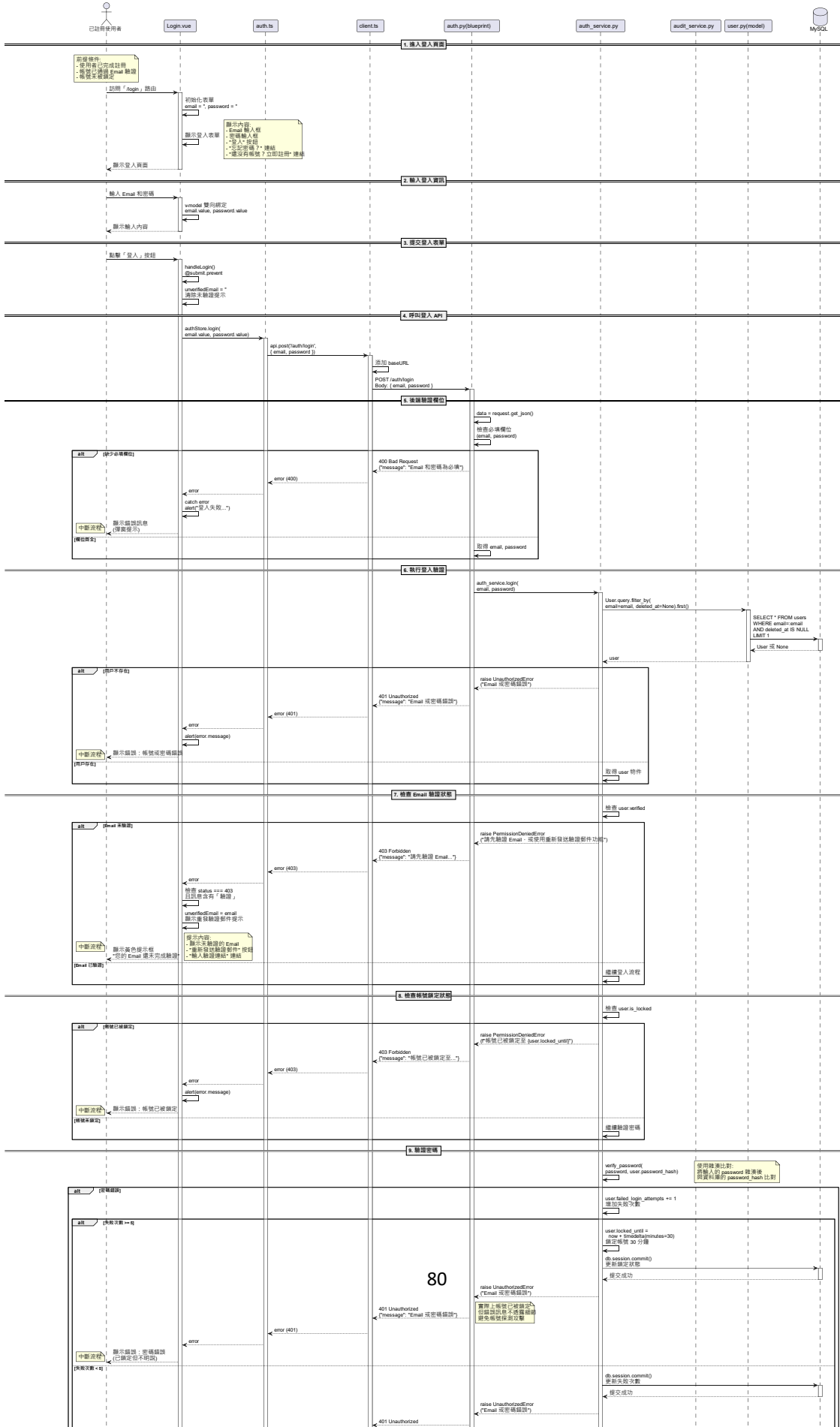
7_1_1: 通知產生與發送（子流程）



通知產生與發送子流程

7_2: 通知中心檢視

使用案例7.2: 使用者登入流程



通知中心檢視

7_3: 通知標記已讀

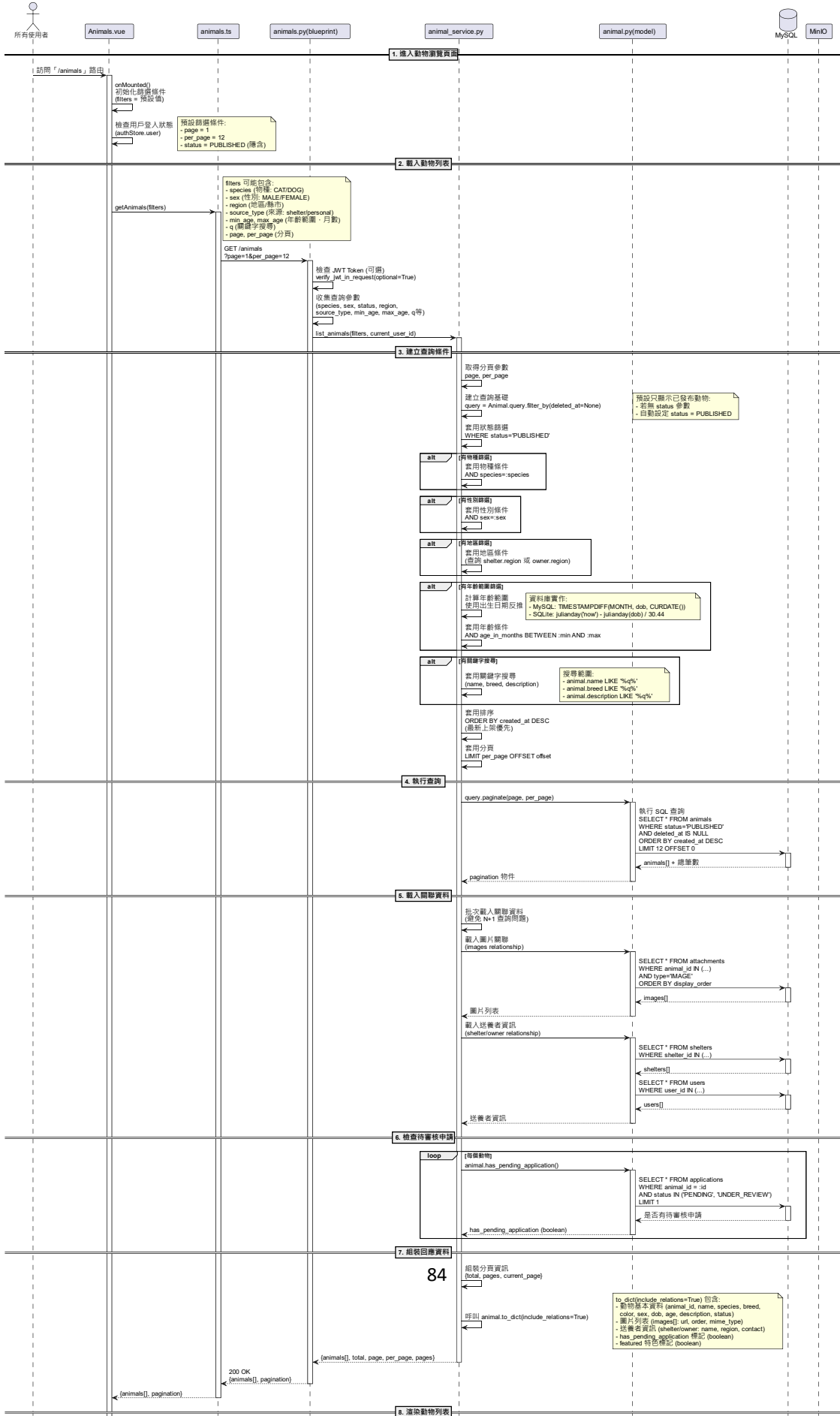
通知標記已讀

6. 物件互動說明

6.1 動物列表查詢流程

循序圖：

使用案例1.1: 動物列表瀏覽



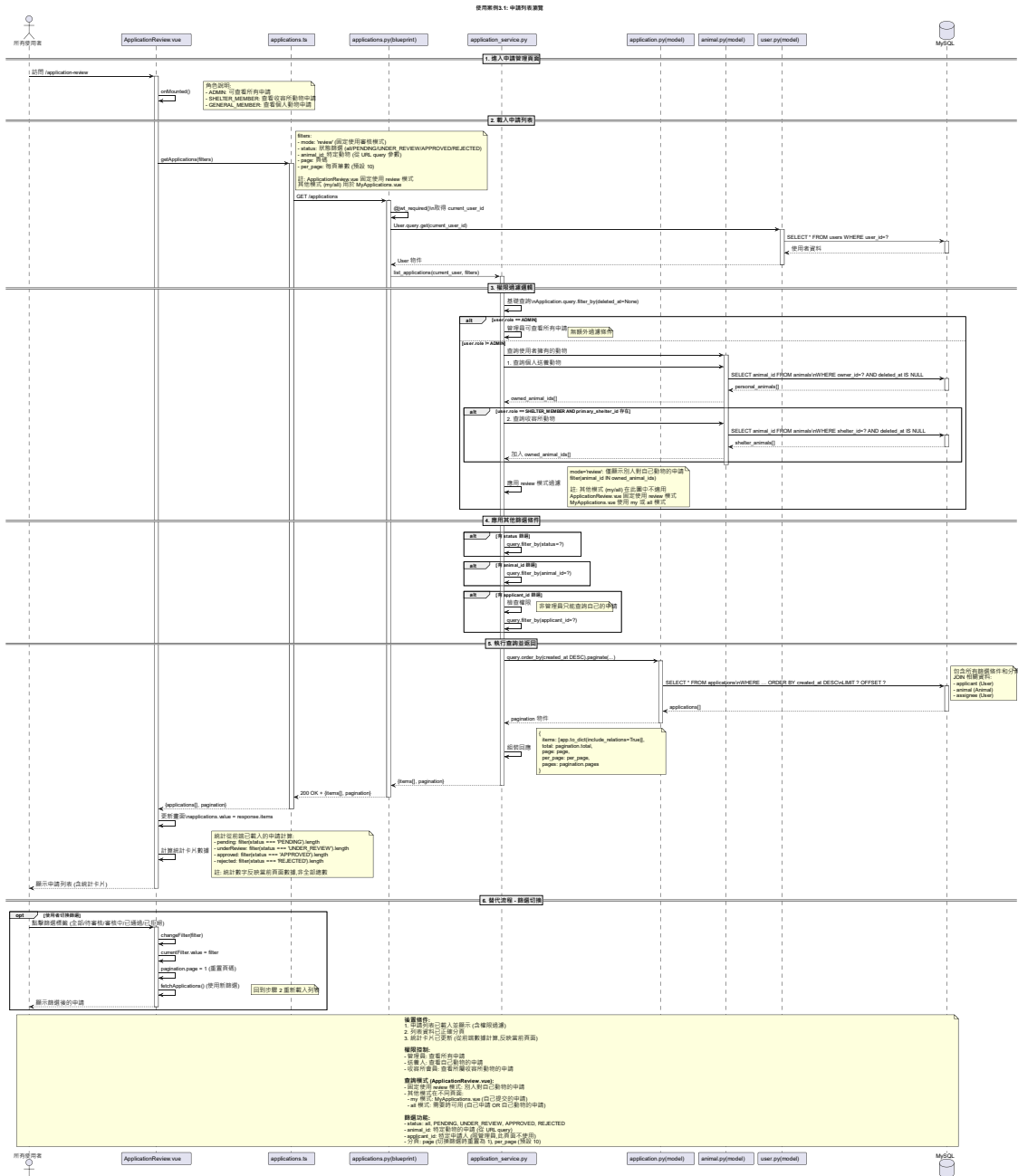
動物列表查詢流程

流程摘要: 1. 使用者進入 /animals 頁面 2. Animals.vue 初始化篩選條件，呼叫 animals.ts 的 getAnimals(filters) 3. animals.ts 發送 GET /animals 請求至後端 4. animals.py (blueprint) 接收請求，驗證 JWT (可選) 5. 呼叫 animal_service.py 的 list_animals(filters, current_user_id) 6. Service 層建立查詢條件： - 預設只顯示 status=PUBLISHED 的動物 - 套用使用者提供的篩選條件（物種、性別、地區等） - 執行分頁查詢 7. 從資料庫取得動物清單 8. 載入關聯的圖片資料（images） 9. 檢查是否有待審核申請（has_pending_application） 10. 回傳 JSON 格式的分頁結果

關鍵物件: - Animals.vue: 前端頁面元件 - animals.ts: API 客戶端 - animals.py: Flask Blueprint (路由處理) - animal_service.py: 業務邏輯層 - Animal model: 資料模型

6.2 領養申請提交流程

循序圖:



領養申請提交流程

流程摘要: 1. 使用者在動物詳情頁點選「提出申請」 2. 顯示申請表單，填寫必要資訊 3. 前端呼叫 createApplication(data, idempotencyKey) 4. 後端檢查：- 使用者角色必須是 GENERAL_MEMBER - 不能對自己的動物提出申請 - 檢查 idempotency_key 避免重複提交 5. 建立 Application 記錄，狀態設為 PENDING 6. 觸發通知服務：- 通知動物擁有者 - 通知相關管理員 7. 記錄稽核日誌 (AuditLog) 8. 回傳新建的申請資料

關鍵驗證規則:

```
# 業務規則檢查
if current_user.role != UserRole.GENERAL_MEMBER:
    raise BusinessException("只有一般會員可以提出申請", 403)

if animal.owner_id == current_user.user_id:
    raise BusinessException("不能對自己的動物提出申請", 400)

# 冪等性檢查
if idempotency_key:
    existing = Application.query.filter_by(
        idempotency_key=idempotency_key
    ).first()
    if existing:
        return existing # 回傳已存在的申請
```

6.3 檔案上傳流程

方式一：直接上傳（後端代理）

Endpoint: POST /uploads/direct

1. 前端將檔案透過 multipart/form-data 上傳至後端
2. 後端生成唯一的 storage_key
3. 後端使用 MinIO client 將檔案上傳至 Object Storage
4. 回傳檔案 URL 與 metadata

方式二：Presigned URL（前端直傳）

Endpoint: POST /uploads/presign

1. 前端請求 presigned URL
 2. 後端生成 presigned URL（有效期 1 小時）
 3. 前端直接透過 presigned URL 上傳至 MinIO
 4. 上傳完成後，前端呼叫 POST /attachments 建立 metadata 記錄
-

6. 物件互動說明

6.1 動物列表查詢流程

參考: [1_1.puml](#)

參與物件: - Animals.vue (前端頁面) - animals.ts (API 客戶端) - animals.py (Flask Blueprint) - animal_service.py (業務邏輯層) - Animal model (資料模型) - MySQL Database - MinIO (圖片儲存)

流程摘要: 1. 初始化階段: - 使用者進入 /animals 頁面 - Animals.vue 在 onMounted() 中初始化篩選條件 - 檢查使用者登入狀態 (可選)

2. API 請求階段:

- 前端呼叫 getAnimals(filters) 傳送篩選參數
- 篩選參數包括: species, sex, region, source_type, age range, keyword 等
- 發送 GET /animals?page=1&per_page=12&...

3. 後端處理階段:

- Blueprint 接收請求, 驗證 JWT token (optional)
- 收集查詢參數並傳遞給 Service 層
- Service 建立查詢基礎: Animal.query.filter_by(deleted_at=None)
- 套用篩選條件:

預設只顯示已發布的動物

```
if not filters.get('status'):
    query = query.filter(Animal.status == AnimalStatus.PUBLISHED)
```

套用物種篩選

```
if filters.get('species'):
    query = query.filter(Animal.species == filters['species'])
```

套用地區篩選

```
if filters.get('region'):
    # 需同時檢查 owner.region 和 shelter.region
    query = query.join(User, Animal.owner_id == User.user_id)
    query = query.filter(
        or_(User.region == filters['region'],
            Shelter.region == filters['region'])
    )
```

套用年齡範圍

```
if filters.get('min_age'):
    # 計算出生日期上限
    max_dob = today - timedelta(days=filters['min_age'] * 30)
    query = query.filter(Animal.dob <= max_dob)
```

4. 關聯資料載入:

- 使用 joinedload() 預載圖片資料, 避免 N+1 查詢
- 檢查每個動物是否有待審核申請

5. 資料轉換與回應:

- 將查詢結果轉換為字典格式 animal.to_dict(include_relations=True)
- 構建分頁回應資料

- 回傳 JSON 格式結果

關鍵優化: - 使用資料庫索引 (species, status, region) - Eager loading 避免 N+1 問題
- 分頁限制每頁最多 100 筆

6.2 動物詳情檢視流程

參考: [1_3.puml](#)

參與物件: - AnimalDetail.vue - animals.ts, medicalRecords.ts - animals.py, medical_records.py (Blueprints) - animal_service.py, medical_record_service.py - Animal, MedicalRecord, User, Shelter models

流程摘要: 1. 頁面初始化: - 使用者從列表點擊動物卡片 - Router 導向 /animals/:id - onMounted() 取得路由參數 animal_id

2. 串行載入資料:

```
async function loadAnimal() {  
  // 1. 載入動物完整資料  
  animal.value = await getAnimal(animalId)  
  
  // 2. 載入醫療紀錄 (前 3 筆摘要)  
  medicalRecords.value = await getMedicalRecords(animalId, { limit: 3 })  
  
  // 3. 載入來源資訊 (飼主或收容所)  
  await loadSourceInfo()  
}
```

3. 動物資料查詢:

- 後端查詢 Animal.query.filter_by(animal_id=:id, deleted_at=None)
- 使用 include_relations=True 載入：
 - 所有圖片 (按 order 排序)
 - 擁有者資訊 (User 或 Shelter)
 - 檢查是否有待審核申請

4. 醫療紀錄查詢:

- 查詢該動物的醫療紀錄
- 按日期降序排序
- 僅回傳已驗證的紀錄 (或擁有者可見所有)

5. 權限檢查:

- 確定當前使用者是否為擁有者

- 決定是否顯示編輯按鈕
- 決定「提出申請」按鈕的可見性：

```
const canApply = computed(() => {
  // 必須登入且為一般會員
  if (!authStore.isAuthenticated || authStore.user?.role !== 'GENERAL_MEMBER') {
    return false
  }
  // 不能對自己的動物申請
  if (animal.value.owner_id === authStore.user.user_id) {
    return false
  }
  // 動物必須是已發布狀態
  return animal.value.status === 'PUBLISHED'
})
```

6. 圖片檢視器：

- 支援點擊放大
- 左右滑動切換
- 顯示圖片順序指示器

互動行為： - 點擊「提出申請」→ 開啟申請表單 modal - 點擊「查看完整醫療紀錄」→ 導向醫療紀錄頁面 - 點擊「編輯」（擁有者）→ 導向編輯表單

6.3 領養申請提交流程

參考： 3_1.puml

參與物件： - AnimalDetail.vue (包含申請表單 modal) - applications.ts - applications.py - application_service.py - notification_service.py - Application, Animal, User models

流程摘要： 1. 開啟申請表單： - 使用者在動物詳情頁點擊「提出申請」 - 顯示 modal，包含申請表單

2. 填寫申請資料：

```
const formData = ref({
  animal_id: animalId,
  type: 'ADOPTION',
  contact_phone: "",
  contact_address: "",
  occupation: "",
  housing_type: "",
  has_experience: false,
```

```

    reason: "",
    notes: ""
  })

```

3. 前端驗證:

```

// 使用 vee-validate + zod
const schema = z.object({
  contact_phone: z.string().regex(/^09\d{8}$/),
  reason: z.string().min(10).max(1000),
  // ... 其他欄位驗證
})

```

4. 提交申請 (冪等性):

```

import { v4 as uuidv4 } from 'uuid'

const idempotencyKey = uuidv4()

try {
  const result = await createApplication(formData.value, idempotencyKey)
  showSuccess('申請已提交')
  router.push('/my-applications')
} catch (error) {
  handleError(error)
}

```

5. 後端業務邏輯檢查:

```

def create_application(applicant, data, idempotency_key):
    # 1. 角色檢查
    if applicant.role != UserRole.GENERAL_MEMBER:
        raise BusinessException("只有一般會員可以提出申請", 403)

    # 2. 冪等性檢查
    if idempotency_key:
        existing = Application.query.filter_by(
            idempotency_key=idempotency_key
        ).first()
        if existing:
            return existing # 直接回傳已存在的申請

    # 3. 取得動物資料
    animal = Animal.query.filter_by(
        animal_id=data['animal_id'],
        deleted_at=None
    ).first()
    if not animal:
        raise NotFoundError("動物不存在")

```

4. 檢查不能對自己的動物申請

```
if animal.owner_id == applicant.user_id:  
    raise BusinessException("不能對自己的動物提出申請", 400)
```

5. 檢查動物狀態

```
if animal.status != AnimalStatus.PUBLISHED:  
    raise BusinessException("只能對已發布的動物提出申請", 400)
```

6. 建立申請記錄

```
application = Application(  
    applicant_id=applicant.user_id,  
    animal_id=animal.animal_id,  
    type=ApplicationType.ADOPTION,  
    status=ApplicationStatus.PENDING,  
    submitted_at=get_naive_taipei_now(),  
    idempotency_key=idempotency_key,  
    **data # 其他申請資料  
)
```

```
db.session.add(application)  
db.session.flush() # 取得 application_id
```

7. 發送通知

```
notification_service.notify_application_submitted(  
    application=application,  
    animal=animal  
)
```

8. 記錄稽核日誌

```
audit_service.log_action(  
    actor_id=applicant.user_id,  
    action='CREATE',  
    resource_type='Application',  
    resource_id=application.application_id  
)
```

```
db.session.commit()  
return application
```

6. 通知觸發:

- 通知動物擁有者: 「您的動物 {name} 收到新的領養申請」
- 通知管理員 (可選): 「新的領養申請待審核」
- 通知方式: 站內通知 + Email (背景任務)

7. 前端導向:

- 申請成功後導向「我的申請」頁面
- 顯示成功訊息
- 申請狀態為 PENDING

錯誤處理: - 400: 驗證失敗、業務規則違反 - 401: 未登入 - 403: 角色權限不足 - 404: 動物不存在

6.4 申請審核流程

參考: [4_2.puml](#)

參與物件: - ApplicationReview.vue (審核頁面) - applications.ts - applications.py - application_service.py - notification_service.py, audit_service.py

流程摘要: 1. 檢視待審核申請: - 動物擁有者或管理員進入審核頁面 - 呼叫 GET /applications?mode=review - 顯示所有待審核的申請列表

2. 檢視申請詳情:

- 點擊申請項目
- 顯示完整申請資料:
 - 申請人基本資料
 - 聯絡方式
 - 居住環境
 - 飼養經驗
 - 領養理由

3. 審核決策:

```
async function reviewApplication(applicationId: number, action: 'approve' | 'reject') {
  const data = {
    action,
    review_notes: reviewNotes.value,
    version: application.value.version // Optimistic locking
  }

  try {
    const result = await reviewApplication(applicationId, data)
    showSuccess(action === 'approve' ? '申請已核准': '申請已拒絕')
    await loadApplications() // 重新載入列表
  } catch (error) {
    if (error.response?.status === 409) {
      showError('申請已被他人更新，請重新整理')
    }
  }
}
```

```
}  
}
```

4. 後端審核邏輯:

```
def review_application(application_id, reviewer, action, review_notes, version):  
    # 1. 取得申請並鎖定 (避免並發衝突)  
    application = Application.query.filter_by(  
        application_id=application_id  
    ).with_for_update().first()  
  
    if not application:  
        raise NotFoundError("申請不存在")  
  
    # 2. 版本檢查 (Optimistic Locking)  
    if application.version != version:  
        raise ConflictError("申請已被更新，請重新整理")  
  
    # 3. 權限檢查  
    animal = application.animal  
    if not (reviewer.is_admin or  
           animal.owner_id == reviewer.user_id or  
           (animal.shelter_id and animal.shelter.primary_account_user_id == reviewer.user  
_id)):  
        raise PermissionDeniedError("無權限審核此申請")  
  
    # 4. 狀態檢查  
    if application.status not in [ApplicationStatus.PENDING, ApplicationStatus.UNDER_  
REVIEW]:  
        raise BusinessException("只能審核待審核的申請")  
  
    # 5. 更新申請狀態  
    if action == 'approve':  
        application.status = ApplicationStatus.APPROVED  
        # 可選：自動將動物狀態改為 ADOPTED  
        # animal.status = AnimalStatus.ADOPTED  
    elif action == 'reject':  
        application.status = ApplicationStatus.REJECTED  
  
    application.reviewed_at = get_naive_taipei_now()  
    application.review_notes = review_notes  
    application.version += 1  
  
    # 6. 發送通知  
    notification_service.notify_application_reviewed(  
        application=application,  
        action=action  
    )
```

```

# 7. 記錄稽核日誌
audit_service.log_action(
    actor_id=reviewer.user_id,
    action='UPDATE',
    resource_type='Application',
    resource_id=application.application_id,
    changes={
        'status': {
            'old': ApplicationStatus.PENDING.value,
            'new': application.status.value
        }
    }
)

db.session.commit()
return application

```

5. 通知申請人:

- 核准：「恭喜！您的領養申請已通過審核」
- 拒絕：「很抱歉，您的領養申請未通過審核」
- 包含審核意見（如有）

並發控制: - 使用 version 欄位實現 Optimistic Locking - 防止多人同時審核同一申請 - 衝突時回傳 409 Conflict

6.5 個人送養發佈流程

參考: [2_1.puml](#)

參與物件: - RehomeForm.vue (送養表單) - FileUploader.vue (檔案上傳元件) - animals.ts, uploads.ts - animals.py, uploads.py - animal_service.py

流程摘要: 1. 進入表單: - 一般會員點擊「發佈送養」 - 檢查登入狀態 - 初始化空白表單或載入草稿

2. 草稿功能:

```

// 從 localStorage 載入草稿
function loadDraft() {
    const draft = localStorage.getItem('rehome_draft')
    if (draft) {
        try {
            formData.value = JSON.parse(draft)
            showInfo('已載入草稿')
        } catch (e) {

```

```

        console.error('載入草稿失敗', e)
      }
    }
  }

  // 自動儲存草稿 (debounce)
  const saveDraft = debounce(() => {
    localStorage.setItem('rehome_draft', JSON.stringify(formData.value))
  }, 1000)

  watch(formData, saveDraft, { deep: true })

```

3. 步驟一：基本資訊：

- 填寫動物名稱、物種、品種
- 選擇性別、出生日期
- 填寫顏色、描述

4. 步驟二：上傳圖片：

```

async function uploadImage(file: File) {
  try {
    // 方式一：直接上傳（後端代理）
    const formData = new FormData()
    formData.append('file', file)

    const response = await uploadDirect(formData)

    images.value.push({
      storage_key: response.storage_key,
      url: response.url,
      order: images.value.length
    })

    showSuccess('圖片上傳成功')
  } catch (error) {
    showError('圖片上傳失敗')
  }
}

```

5. 步驟三：醫療資訊（可選）：

- 填寫絕育狀態
- 填寫疫苗接種記錄
- 上傳醫療證明

6. 提交送養：


```

async function submitRehome() {
  // 驗證表單
  const isValid = await validate()
  if (!isValid) return

  try {
    // 建立動物資料 (狀態: DRAFT)
    const animalData = {
      ...formData.value,
      status: 'DRAFT'
    }

    const result = await createAnimal(animalData)
    const animalId = result.animal.animal_id

    // 關聯圖片
    for (const [index, image] of images.value.entries()) {
      await addAnimalImage(animalId, {
        storage_key: image.storage_key,
        url: image.url,
        order: index
      })
    }

    // 提交審核 (將狀態改為 SUBMITTED)
    await submitAnimalForReview(animalId)

    // 清除草稿
    localStorage.removeItem('rehome_draft')

    showSuccess('送養資訊已提交審核')
    router.push('/my-rehomes')

  } catch (error) {
    handleError(error)
  }
}

```

7. 後端處理:

- 建立 Animal 記錄
- 關聯 AnimalImage 記錄
- 狀態轉換: DRAFT → SUBMITTED
- 通知管理員待審核

狀態機:

DRAFT (草稿)

- [提交審核] → SUBMITTED (待審核)
- [管理員核准] → PUBLISHED (已發布)
- [領養成功] → ADOPTED (已領養)
- [下架] → RETIRED (已下架)

6.6 收容所批次匯入流程

參考: [5_1.puml](#)

參與物件: - ShelterBatch.vue - shelters.ts, jobs.ts - shelters.py, jobs.py - Celery Worker - animal_tasks.py

流程摘要: 1. 上傳檔案: - 收容所管理員選擇 CSV/JSON 檔案 - 前端驗證檔案格式與大小 - 上傳至後端

2. 建立背景任務:

```
@shelters_bp.route('/<int:shelter_id>/animals/batch', methods=['POST'])
@jwt_required()
def batch_import_animals(shelter_id):
    # 1. 權限檢查
    # 2. 接收檔案或 JSON 資料
    data = request.get_json()

    # 3. 建立 Job 記錄
    job = Job(
        type='import_animals',
        status=JobStatus.PENDING,
        payload={
            'shelter_id': shelter_id,
            'animals': data['animals']
        },
        created_by=current_user_id
    )
    db.session.add(job)
    db.session.commit()

    # 4. 將任務加入 Celery 佇列
    from app.tasks.animal_tasks import import_animals_task
    import_animals_task.delay(job.job_id)

    # 5. 立即回傳 202 Accepted
    return jsonify({
        'message': '批次匯入任務已建立',
        'job_id': job.job_id,
```

```
        'status': 'PENDING'
    }), 202
```

3. Celery Worker 處理:

```
@celery.task(bind=True)
def import_animals_task(self, job_id):
    job = Job.query.get(job_id)
    job.status = JobStatus.RUNNING
    job.started_at = get_naive_taipei_now()
    db.session.commit()

    try:
        animals_data = job.payload['animals']
        shelter_id = job.payload['shelter_id']

        results = {
            'total': len(animals_data),
            'success': 0,
            'failed': 0,
            'errors': []
        }

        for index, animal_data in enumerate(animals_data):
            try:
                # 驗證資料
                validate_animal_data(animal_data)

                # 建立動物
                animal = Animal(
                    shelter_id=shelter_id,
                    status=AnimalStatus.DRAFT,
                    **animal_data
                )
                db.session.add(animal)
                db.session.flush()

                results['success'] += 1

                # 更新進度
                self.update_state(
                    state='PROGRESS',
                    meta={
                        'current': index + 1,
                        'total': results['total']
                    }
                )

            except Exception as e:
```

```

        results['failed'] += 1
        results['errors'].append(f"第 {index+1} 行: {str(e)}")

    db.session.commit()

    # 更新 Job 狀態
    job.status = JobStatus.SUCCEEDED
    job.result_summary = results

except Exception as e:
    job.status = JobStatus.FAILED
    job.result_summary = {'error': str(e)}
    db.session.rollback()

finally:
    job.finished_at = get_naive_taipei_now()
    job.attempts += 1
    db.session.commit()

```

4. 前端輪詢進度:

```

async function pollJobStatus(jobId: number) {
    const maxAttempts = 60 // 最多輪詢5分鐘
    let attempts = 0

    const poll = async () => {
        try {
            const job = await getJob(jobId)

            if (job.status === 'SUCCEEDED') {
                showSuccess(`批次匯入完成: 成功 ${job.result_summary.success}，失敗 ${job.result_summary.failed}`)
                await loadAnimals() // 重新載入動物列表
                return
            }

            if (job.status === 'FAILED') {
                showError('批次匯入失敗')
                return
            }

            // 顯示進度
            if (job.progress) {
                progressPercentage.value = job.progress.percentage
            }

            // 繼續輪詢
            attempts++

```

```

    if (attempts < maxAttempts) {
      setTimeout(poll, 5000) // 每 5 秒輪詢一次
    }

    } catch (error) {
      console.error('輪詢失敗', error)
    }
  }

  poll()
}

```

CSV 格式範例:

```

name,species,breed,sex,dob,description
小黑,DOG,拉布拉多,MALE,2023-01-15,友善活潑的狗狗
小花,CAT,米克斯,FEMALE,2024-03-10,溫柔親人的貓咪

```

6.7 通知系統流程

參考: [7_1.puml](#), [7_2.puml](#)

通知觸發時機: 1. 申請提交 → 通知動物擁有者 2. 申請審核 → 通知申請人 3. 動物發布 → 通知關注該收容所的會員（未來功能） 4. 系統公告 → 通知所有使用者

通知產生流程:

```

# notification_service.py
def notify_application_submitted(application, animal):
    # 通知動物擁有者
    notification = Notification(
        recipient_id=animal.owner_id,
        actor_id=application.applicant_id,
        type='application_submitted',
        payload={
            'application_id': application.application_id,
            'animal_id': animal.animal_id,
            'animal_name': animal.name,
            'applicant_username': application.applicant.username,
            'message': f'{application.applicant.username} 對您的動物 {animal.name} 提出了領養申請'
        }
    )
    db.session.add(notification)
    db.session.commit()

# 發送 Email (背景任務)

```

```
from app.tasks.email_tasks import send_notification_email
send_notification_email.delay(notification.notification_id)
```

前端通知中心:

```
// NotificationCenter.vue
const { data: notifications, refetch } = useQuery({
  queryKey: ['notifications'],
  queryFn: () => getNotifications({ unread_only: false }),
  refetchInterval: 30000 // 每 30 秒自動重新整理
})
```

```
// NotificationBell.vue (導航欄通知圖示)
const { data: unreadCount } = useQuery({
  queryKey: ['notifications', 'unread-count'],
  queryFn: getUnreadNotificationCount,
  refetchInterval: 10000 // 每 10 秒檢查未讀數量
})
```

標記已讀:

```
async function markAsRead(notificationId: number) {
  await markNotificationRead(notificationId)
  await refetch() // 重新整理列表
}
```

7. 資料驗證規則

7.1 後端驗證 (Python/Flask)

動物資料驗證:

```
# animal_service.py
def validate_animal_data(data):
    errors = []

    # 必填欄位
    if not data.get('species'):
        errors.append('物種為必填')

    # 年齡邏輯檢查
    if data.get('dob'):
        dob = datetime.strptime(data['dob'], '%Y-%m-%d').date()
        if dob > date.today():
            errors.append('出生日期不能晚於今天')

    # 狀態轉換驗證
```

```

if data.get('status') == 'PUBLISHED':
    if not data.get('name') or not data.get('description'):
        errors.append('發布需要完整的名稱與描述')

if errors:
    raise ValidationError('; '.join(errors))

```

申請資料驗證:

```

# application_service.py
def validate_application_data(data):
    required_fields = ['animal_id', 'contact_phone', 'reason']

    for field in required_fields:
        if not data.get(field):
            raise ValidationError(f'{field} 為必填欄位')

    # 電話號碼格式
    if not re.match(r'^09\d{8}$', data['contact_phone']):
        raise ValidationError('電話號碼格式不正確')

```

7.2 前端驗證 (TypeScript/Vue)

使用 **vee-validate + zod**:

```

import { z } from 'zod'

// 動物表單驗證 schema
export const animalSchema = z.object({
  name: z.string().min(1, '名稱為必填').max(200),
  species: z.enum(['CAT', 'DOG'], { required_error: '請選擇物種' }),
  sex: z.enum(['MALE', 'FEMALE', 'UNKNOWN']).optional(),
  dob: z.string().optional(),
  description: z.string().max(5000, '描述不可超過 5000 字').optional(),
})

// 申請表單驗證 schema
export const applicationSchema = z.object({
  animal_id: z.number(),
  contact_phone: z.string()
    .regex(/^09\d{8}$/, '請輸入有效的手機號碼'),
  reason: z.string()
    .min(10, '請至少輸入 10 個字的領養理由')
    .max(1000),
  has_experience: z.boolean(),
})

```

8. 錯誤處理

8.1 HTTP 狀態碼規範

狀態碼	說明	使用情境
200 OK	成功	GET, PATCH 成功
201 Created	已建立	POST 建立資源成功
400 Bad Request	錯誤請求	驗證失敗、參數錯誤
401 Unauthorized	未認證	缺少或無效的 JWT token
403 Forbidden	無權限	角色權限不足
404 Not Found	找不到資源	資源不存在或已刪除
409 Conflict	衝突	重複註冊、資源衝突
500 Internal Server Error	伺服器錯誤	未預期的系統錯誤

8.2 錯誤回應格式

標準錯誤格式:

```
{
  "message": "錯誤訊息描述",
  "error_code": "VALIDATION_ERROR",
  "details": {
    "field": "email",
    "reason": "Email 格式不正確"
  }
}
```

前端錯誤處理:

```
// api/client.ts
api.interceptors.response.use(
  response => response,
  error => {
    if (error.response?.status === 401) {
      // Token 過期，導向登入頁
      router.push('/login')
    } else if (error.response?.status === 403) {
      // 權限不足
      showError('您沒有執行此操作的權限')
    }
    return Promise.reject(error)
  }
)
```



```
}  
)
```

9. 並發控制

9.1 Optimistic Locking

使用 **version** 欄位防止並發更新衝突

Application 模型:

```
class Application(db.Model):  
    version = db.Column(db.Integer, default=1, nullable=False)
```

更新邏輯:

```
def review_application(application_id, action, version):  
    application = Application.query.filter_by(  
        application_id=application_id  
    ).with_for_update().first()  
  
    # 版本檢查  
    if application.version != version:  
        raise ConflictError('申請已被他人更新，請重新整理')  
  
    # 執行更新  
    application.status = new_status  
    application.version += 1  
    db.session.commit()
```

9.2 Idempotency (冪等性)

使用 **Idempotency-Key** 防止重複提交

申請建立:

```
@applications_bp.route("/", methods=['POST'])  
@jwt_required()  
def create_application():  
    idempotency_key = request.headers.get('Idempotency-Key')  
  
    if idempotency_key:  
        existing = Application.query.filter_by(  
            idempotency_key=idempotency_key  
        ).first()
```

```
if existing:
    # 回傳已存在的申請
    return jsonify({
        'message': '申請已存在',
        'application': existing.to_dict()
    }), 200
```

建立新申請...

前端實作:

```
import { v4 as uuidv4 } from 'uuid'

export async function createApplication(data: CreateApplicationData) {
    const idempotencyKey = uuidv4()

    const response = await api.post('/applications', data, {
        headers: {
            'Idempotency-Key': idempotencyKey
        }
    })

    return response.data
}
```

10. 效能考量

10.1 資料庫查詢優化

- 使用索引: email, animal_id, applicant_id, status
- Eager Loading: 使用 joinedload() 預載關聯資料
- 分頁查詢: 限制每頁最大 100 筆

10.2 快取策略

- 前端使用 @tanstack/vue-query 快取 API 回應
- 圖片 URL 使用 CDN (MinIO + Nginx)
- 靜態資源快取 (瀏覽器快取 header)

10.3 非同步處理

- 批次上傳使用 Celery 背景任務
 - Email 發送透過 Celery worker
 - 長時間任務回傳 202 + jobId
-

10. 效能考量

10.1 資料庫查詢優化

索引策略:

-- 常用查詢欄位建立索引

```
CREATE INDEX idx_animals_status ON animals(status);
CREATE INDEX idx_animals_species ON animals(species);
CREATE INDEX idx_animals_shelter_id ON animals(shelter_id);
CREATE INDEX idx_animals_owner_id ON animals(owner_id);
CREATE INDEX idx_animals_created_at ON animals(created_at);
```

```
CREATE INDEX idx_applications_status ON applications(status);
CREATE INDEX idx_applications_applicant_id ON applications(applicant_id);
CREATE INDEX idx_applications_animal_id ON applications(animal_id);
```

```
CREATE INDEX idx_notifications_recipient_id_read ON notifications(recipient_id, read);
CREATE INDEX idx_users_email ON users(email);
```

查詢優化範例:

✗ 不好的做法：N+1 查詢問題

```
animals = Animal.query.filter_by(status='PUBLISHED').all()
for animal in animals:
    print(animal.images) # 每次都觸發一次查詢
```

✓ 好的做法：使用 joinedload 預載關聯

```
from sqlalchemy.orm import joinedload
```

```
animals = Animal.query.filter_by(status='PUBLISHED')\
    .options(joinedload(Animal.images))\
    .options(joinedload(Animal.owner))\
    .all()
```

分頁查詢:

```
def list_animals_paginated(page=1, per_page=20):
```

限制 per_page 最大值

```
per_page = min(per_page, 100)
```

```
pagination = Animal.query.filter_by(
    status=AnimalStatus.PUBLISHED,
    deleted_at=None
).order_by(
    Animal.created_at.desc()
).paginate(
    page=page,
    per_page=per_page,
```

```

    error_out=False
)

return {
    'animals': [a.to_dict() for a in pagination.items],
    'total': pagination.total,
    'pages': pagination.pages,
    'page': page,
    'per_page': per_page
}

```

10.2 快取策略

前端快取（@tanstack/vue-query）：

```

// 動物列表 - 快取 5 分鐘
const { data: animals } = useQuery({
  queryKey: ['animals', filters],
  queryFn: () => getAnimals(filters),
  staleTime: 5 * 60 * 1000, // 5 分鐘內視為新鮮
  cacheTime: 10 * 60 * 1000 // 10 分鐘後清除快取
})

```

```

// 動物詳情 - 快取 10 分鐘
const { data: animal } = useQuery({
  queryKey: ['animal', animalId],
  queryFn: () => getAnimal(animalId),
  staleTime: 10 * 60 * 1000
})

```

```

// 通知數量 - 頻繁更新
const { data: unreadCount } = useQuery({
  queryKey: ['notifications', 'unread-count'],
  queryFn: getUnreadNotificationCount,
  refetchInterval: 10000, // 每 10 秒重新整理
  staleTime: 5000
})

```

後端快取（Redis - 未來擴充）：

```

# 快取熱門動物列表
@cache.memoize(timeout=300) # 5 分鐘
def get_featured_animals():
    return Animal.query.filter_by(
        status=AnimalStatus.PUBLISHED,
        featured=True
    ).limit(10).all()

```

```
# 快取收容所資訊
@cache.memoize(timeout=3600) # 1 小時
def get_shelter_info(shelter_id):
    return Shelter.query.get(shelter_id)
```

靜態資源快取:

```
# Nginx 設定
location ~* \.(jpg|jpeg|png|gif|ico|css|js)$ {
    expires 30d;
    add_header Cache-Control "public, immutable";
}
```

10.3 圖片處理與 CDN

圖片上傳優化:

```
// 前端壓縮圖片
async function compressImage(file: File): Promise<Blob> {
    return new Promise((resolve, reject) => {
        const reader = new FileReader()
        reader.onload = (e) => {
            const img = new Image()
            img.onload = () => {
                const canvas = document.createElement('canvas')
                const ctx = canvas.getContext('2d')

                // 限制最大寬度 1920px
                const maxWidth = 1920
                const scale = Math.min(1, maxWidth / img.width)

                canvas.width = img.width * scale
                canvas.height = img.height * scale

                ctx.drawImage(img, 0, 0, canvas.width, canvas.height)

                canvas.toBlob((blob) => {
                    resolve(blob)
                }, 'image/jpeg', 0.85) // 85% 品質
            }
            img.src = e.target.result as string
        }
        reader.readAsDataURL(file)
    })
}
```

MinIO + CDN 架構:

使用者 → Nginx (CDN) → MinIO (Object Storage)

↓
快取層 (304 Not Modified)

圖片 URL 生成:

```
def get_image_url(storage_key):
    # 開發環境：直接使用 MinIO URL
    if Config.ENV == 'development':
        return f"{Config.MINIO_PUBLIC_ENDPOINT}/{Config.MINIO_BUCKET}/{storage_key}"

    # 生產環境：使用 CDN URL
    return f"{Config.CDN_URL}/{storage_key}"
```

10.4 非同步處理

背景任務類型: 1. **Email 發送** - 立即執行，失敗可重試 2. **批次匯入** - 長時間執行，回傳 job_id 3. **報表生成** - 長時間執行，完成後發通知 4. **圖片處理** - 縮圖生成、格式轉換

Celery 設定:

```
# celery.py
from celery import Celery

celery = Celery('app')
celery.config_from_object('config.CeleryConfig')

# 定義任務佇列
celery.conf.task_routes = {
    'app.tasks.email_tasks.*': {'queue': 'emails'},
    'app.tasks.animal_tasks.*': {'queue': 'animals'},
    'app.tasks.report_tasks.*': {'queue': 'reports'},
}

# 重試策略
celery.conf.task_annotations = {
    'app.tasks.email_tasks.send_email': {
        'rate_limit': '100/m', # 每分鐘最多 100 封
        'max_retries': 3,
        'default_retry_delay': 60 # 失敗後 60 秒重試
    }
}
```

任務範例:

```
# email_tasks.py
@celery.task(bind=True, max_retries=3)
def send_notification_email(self, notification_id):
    try:
        notification = Notification.query.get(notification_id)
        recipient = notification.recipient

        send_email(
            to=recipient.email,
            subject=f'新通知: {notification.type}',
            template='notification.html',
            context={'notification': notification}
        )

    except SMTPException as e:
        # 暫時性錯誤，重試
        raise self.retry(exc=e, countdown=60)

    except Exception as e:
        # 永久性錯誤，記錄但不重試
        logger.error(f'Failed to send email for notification {notification_id}: {e}')
```

前端輪詢 vs WebSocket:

```
// 方案一：輪詢（目前實作）
const pollInterval = setInterval(async () => {
    const job = await getJob(jobId)
    if (job.status === 'SUCCEEDED' || job.status === 'FAILED') {
        clearInterval(pollInterval)
        handleJobComplete(job)
    }
}, 5000)

// 方案二：WebSocket（未來擴充）
const socket = new WebSocket('ws://api.example.com/ws')
socket.addEventListener('message', (event) => {
    const data = JSON.parse(event.data)
    if (data.type === 'job_update' && data.job_id === jobId) {
        handleJobUpdate(data)
    }
})
```

10.5 Rate Limiting (速率限制)

後端 Rate Limiting:

```

from flask_limiter import Limiter
from flask_limiter.util import get_remote_address

limiter = Limiter(
    app,
    key_func=get_remote_address,
    default_limits=["200 per day", "50 per hour"]
)

# 針對特定端點設定
@animals_bp.route("", methods=['POST'])
@limiter.limit("10 per hour") # 每小時最多建立 10 個動物
@jwt_required()
def create_animal():
    pass

@applications_bp.route("", methods=['POST'])
@limiter.limit("5 per hour") # 每小時最多提交 5 個申請
@jwt_required()
def create_application():
    pass

```

11. 安全性考量

11.1 認證與授權

JWT Token 管理:

```

# config.py
JWT_SECRET_KEY = os.getenv('JWT_SECRET_KEY')
JWT_ACCESS_TOKEN_EXPIRES = timedelta(hours=1)
JWT_REFRESH_TOKEN_EXPIRES = timedelta(days=30)

```

密碼安全:

```

from werkzeug.security import generate_password_hash, check_password_hash

```

註冊時

```

password_hash = generate_password_hash(password, method='pbkdf2:sha256')

```

登入時

```

if not check_password_hash(user.password_hash, password):
    raise UnauthorizedError('密碼錯誤')

```

RBAC 權限檢查:


```

def check_permission(user, action, resource):
    # 管理員擁有所有權限
    if user.role == UserRole.ADMIN:
        return True

    # 檢查資源擁有權
    if action in ['UPDATE', 'DELETE']:
        if resource.owner_id == user.user_id:
            return True
        if hasattr(resource, 'created_by') and resource.created_by == user.user_id:
            return True

    # 收容所成員權限
    if user.role == UserRole.SHELTER_MEMBER:
        if hasattr(resource, 'shelter_id') and resource.shelter_id == user.primary_shelter_id:
            return True

    return False

```

11.2 輸入驗證

SQL Injection 防護:

```

# ✓ 使用 ORM，自動處理參數化查詢
animals = Animal.query.filter_by(name=user_input).all()

# ✗ 避免直接拼接 SQL
# query = f"SELECT * FROM animals WHERE name = '{user_input}'" # 危險！

```

XSS 防護:

```

// 前端：自動轉義 (Vue 預設行為)
<template>
  <div>{{ animal.description }}</div> // 自動轉義
  <div v-html="sanitizedHtml"></div> // 需手動淨化
</template>

// 淨化 HTML
import DOMPurify from 'dompurify'

const sanitizedHtml = computed(() => {
  return DOMPurify.sanitize(animal.value.description)
})

```

CSRF 防護:

```
# Flask-WTF CSRF Protection
from flask_wtf.csrf import CSRFProtect

csrf = CSRFProtect(app)

# JWT 不需要 CSRF token (存在 localStorage)
# 但 Cookie-based auth 需要
```

11.3 檔案上傳安全

檔案類型驗證:

```
ALLOWED_IMAGE_TYPES = {'image/jpeg', 'image/png', 'image/gif', 'image/webp'}
ALLOWED_DOCUMENT_TYPES = {'application/pdf'}
MAX_FILE_SIZE = 10 * 1024 * 1024 # 10MB
```

```
def validate_file(file, allowed_types):
    # 檢查 MIME type
    if file.mimetype not in allowed_types:
        raise ValidationError(f'不支援的檔案類型: {file.mimetype}')

    # 檢查檔案大小
    file.seek(0, 2) # 移到檔案末尾
    size = file.tell()
    file.seek(0) # 回到開頭

    if size > MAX_FILE_SIZE:
        raise ValidationError(f'檔案過大: {size} bytes')

    # 檢查檔案內容 (magic bytes)
    header = file.read(512)
    file.seek(0)

    if not is_valid_image(header):
        raise ValidationError('檔案內容不符合宣告的類型')
```

安全的檔名處理:

```
import uuid
import os

def generate_safe_filename(original_filename):
    # 取得副檔名
    ext = os.path.splitext(original_filename)[1].lower()

    # 生成 UUID 檔名
```

```
safe_name = f"{uuid.uuid4()}{ext}"
```

```
return safe_name
```

11.4 敏感資料處理

環境變數管理:

.env (不應提交到 git)

```
DATABASE_URL=postgresql://user:pass@localhost/db
```

```
JWT_SECRET_KEY=your-secret-key-here
```

```
MINIO_ACCESS_KEY=minioadmin
```

```
MINIO_SECRET_KEY=minioadmin
```

config.py

```
import os
```

```
from dotenv import load_dotenv
```

```
load_dotenv()
```

```
class Config:
```

```
    DATABASE_URL = os.getenv('DATABASE_URL')
```

```
    JWT_SECRET_KEY = os.getenv('JWT_SECRET_KEY')
```

敏感欄位遮罩:

```
def to_dict(self, include_sensitive=False):
```

```
    data = {
```

```
        'user_id': self.user_id,
```

```
        'email': self.email,
```

```
        'username': self.username,
```

```
    }
```

```
    if include_sensitive:
```

```
        data.update({
```

```
            'phone_number': self.phone_number,
```

```
            'address': self.address,
```

```
        })
```

```
    else:
```

```
        # 遮罩敏感資料
```

```
        data['phone_number'] = self._mask_phone(self.phone_number)
```

```
    return data
```

```
def _mask_phone(self, phone):
```

```
    if not phone or len(phone) < 4:
```

```
        return '****'
```

```
    return phone[:4] + '****'
```

12. 測試策略

12.1 後端測試

單元測試範例:

```
# tests/unit/test_animal_service.py
import pytest
from app.services.animal_service import animal_service
from app.models import Animal, User, UserRole

def test_create_animal_success(db_session):
    # Arrange
    user = User(email='test@example.com', role=UserRole.GENERAL_MEMBER)
    db_session.add(user)
    db_session.commit()

    data = {
        'name': '小白',
        'species': 'DOG',
        'breed': '混種犬'
    }

    # Act
    animal = animal_service.create_animal(user, data)

    # Assert
    assert animal.name == '小白'
    assert animal.species == Species.DOG
    assert animal.owner_id == user.user_id
    assert animal.status == AnimalStatus.DRAFT

def test_create_animal_validation_error(db_session):
    user = User(email='test@example.com')
    data = {} # 缺少必填欄位

    with pytest.raises(ValidationError):
        animal_service.create_animal(user, data)
```

整合測試範例:

```
# tests/integration/test_animals_api.py
def test_list_animals_api(client, auth_headers):
    # 建立測試資料
    create_test_animals(count=5)

    # 發送 API 請求
```

```

response = client.get('/animals?page=1&per_page=10')

assert response.status_code == 200
data = response.get_json()
assert len(data['animals']) == 5
assert data['total'] == 5

def test_create_application_requires_auth(client):
    response = client.post('/applications', json={
        'animal_id': 1
    })

    assert response.status_code == 401

```

12.2 前端測試

E2E 測試範例 (Playwright):

```

// tests/e2e/specs/critical-paths/rehome-publishing.spec.ts
import { test, expect } from '@playwright/test'

test.describe('送養發佈流程', () => {
  test('完整發佈流程', async ({ page }) => {
    // 1. 登入
    await page.goto('/login')
    await page.fill('[name="email"]', 'test@example.com')
    await page.fill('[name="password"]', 'password123')
    await page.click('button[type="submit"]')

    // 2. 進入送養表單
    await page.click('text=發佈送養')
    await expect(page).toHaveURL('/rehome/new')

    // 3. 填寫基本資訊
    await page.fill('[name="name"]', '小白')
    await page.selectOption('[name="species"]', 'DOG')
    await page.fill('[name="breed"]', '混種犬')
    await page.selectOption('[name="sex"]', 'MALE')

    // 4. 上傳圖片
    await page.setInputFiles('input[type="file"]', 'tests/fixtures/dog.jpg')
    await page.waitForSelector('img[alt="上傳的圖片"]')

    // 5. 填寫描述
    await page.fill('[name="description"]', '活潑親人的狗狗')

```

```
// 6. 提交
await page.click("button:has-text('提交審核')")

// 7. 驗證結果
await expect(page).toHaveURL('/my-rehomes')
await expect(page.locator('text=送養資訊已提交審核')).toBeVisible()
})
})
```

13. 部署架構

13.1 Docker Compose (開發環境)

```
# docker-compose.yml
version: '3.8'

services:
  backend:
    build: ./backend
    ports:
      - "5000:5000"
    environment:
      - DATABASE_URL=postgresql://user:pass@db:5432/petadopt
      - REDIS_URL=redis://redis:6379/0
      - MINIO_ENDPOINT=minio:9000
    depends_on:
      - db
      - redis
      - minio

  frontend:
    build: ./frontend
    ports:
      - "5173:5173"
    environment:
      - VITE_API_BASE_URL=http://localhost:5000

  db:
    image: postgres:15
    environment:
      POSTGRES_DB: petadopt
      POSTGRES_USER: user
      POSTGRES_PASSWORD: pass
    volumes:
      - postgres_data:/var/lib/postgresql/data

  redis:
```

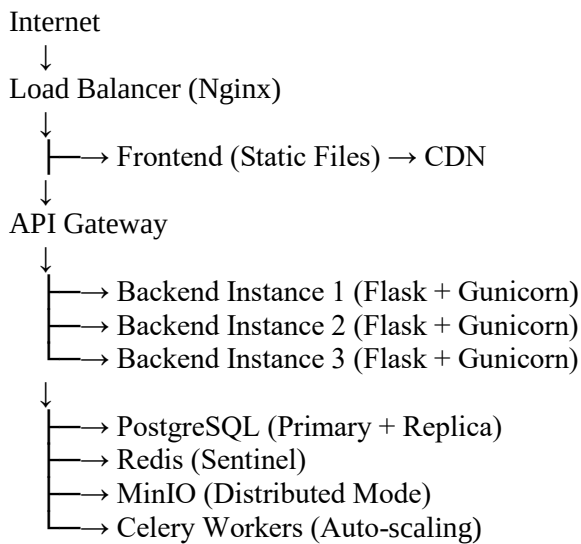
```
image: redis:7-alpine
ports:
  - "6379:6379"

minio:
image: minio/minio
command: server /data --console-address ":9001"
ports:
  - "9000:9000"
  - "9001:9001"
environment:
  MINIO_ROOT_USER: minioadmin
  MINIO_ROOT_PASSWORD: minioadmin
volumes:
  - minio_data:/data

celery-worker:
build: ./backend
command: celery -A app.celery worker --loglevel=info
depends_on:
  - redis
  - db

volumes:
  postgres_data:
  minio_data:
```

13.2 生產環境架構



附錄 A: 完整 API 端點列表

Authentication

方法	端點	描述	認證	權限
POST	/auth/register	使用者註冊	No	-
POST	/auth/login	使用者登入	No	-
POST	/auth/refresh	刷新 access token	Yes (Refresh)	-
GET	/auth/me	取得當前使用者資訊	Yes	-
POST	/auth/logout	登出	Yes	-
GET	/auth/verify	Email 驗證	No	-
POST	/auth/forgot-password	忘記密碼	No	-
POST	/auth/reset-password	重設密碼	No	-

Animals

方法	端點	描述	認證	權限
GET	/animals	動物列表（公開）	Optional	-
GET	/animals/:id	動物詳情	Optional	-
POST	/animals	建立動物	Yes	GENERAL_MEMBER, SHELTER_MEMBER, ADMIN
PATCH	/animals/:id	更新動物	Yes	Owner, Admin
DELETE	/animals/:id	刪除動物（軟刪除）	Yes	Owner, Admin
POST	/animals/:id/images	新增動物圖片	Yes	Owner, Admin
DELETE	/animals/:id/images/:image_id	刪除動物圖片	Yes	Owner, Admin
PATCH	/animals/:id/images/reorder	重新排序圖片	Yes	Owner, Admin
POST	/animals/:id/submit	提交審核	Yes	Owner
POST	/animals/:id/publish	發布動物	Yes	Admin
POST	/animals/:id/retire	下架動物	Yes	Owner, Admin

方法	端點	描述	認證	權限
POST	/animals/:id/reject	拒絕動物	Yes	Admin

Applications

方法	端點	描述	認證	權限
GET	/applications	申請列表	Yes	-
GET	/applications/:id	申請詳情	Yes	Applicant, Owner, Admin
POST	/applications	建立申請	Yes	GENERAL_MEMBER
POST	/applications/:id/review	審核申請	Yes	Owner, Admin
POST	/applications/:id/assign	指派審核人	Yes	Owner, Admin
POST	/applications/:id/withdraw	撤回申請	Yes	Applicant

Medical Records

方法	端點	描述	認證	權限
GET	/medical-records/animals	可管理的動物列表	Yes	-
GET	/medical-records/animals/:id/medical-records	動物醫療紀錄列表	Optional	-
POST	/medical-records/animals/:id/medical-records	建立醫療紀錄	Yes	Owner, Admin
PATCH	/medical-records/:id	更新醫療紀錄	Yes	Creator, Admin
DELETE	/medical-records/:id	刪除醫療紀錄	Yes	Creator, Admin

Shelters

方法	端點	描述	認證	權限
GET	/shelters	收容所列表（公開）	No	-
GET	/shelters/:id	收容所詳情	No	-
POST	/shelters	建立收容所	Yes	SHELTER_MEMBER, ADMIN

方法	端點	描述	認證	權限
PATCH	/shelters/:id	更新收容所	Yes	Manager, Admin
POST	/shelters/:id/animals/batch	批次匯入動物	Yes	Manager, Admin
GET	/shelters/:id/animals	收容所動物列表	No	-
GET	/shelters/:id/statistics	收容所統計資料	No	-

Notifications

方法	端點	描述	認證	權限
GET	/notifications	通知列表	Yes	-
GET	/notifications/unread-count	未讀通知數量	Yes	-
POST	/notifications/:id/read	標記已讀	Yes	Recipient
POST	/notifications/read-all	全部標記已讀	Yes	-
DELETE	/notifications/:id	刪除通知	Yes	Recipient

Jobs

方法	端點	描述	認證	權限
GET	/jobs	任務列表	Yes	-
GET	/jobs/:id	任務詳情	Yes	Creator, Admin

Uploads

方法	端點	描述	認證	權限
POST	/uploads/direct	直接上傳檔案	Yes	-
POST	/uploads/presign	取得預簽名 URL	Yes	-
POST	/attachments	建立附件記錄	Yes	-

Users

方法	端點	描述	認證	權限
GET	/users/:id	使用者公開資料	No	-
PATCH	/users/:id	更新使用者資料	Yes	Self, Admin
POST	/users/:id/change-password	變更密碼	Yes	Self

Admin

方法	端點	描述	認證	權限
----	----	----	----	----

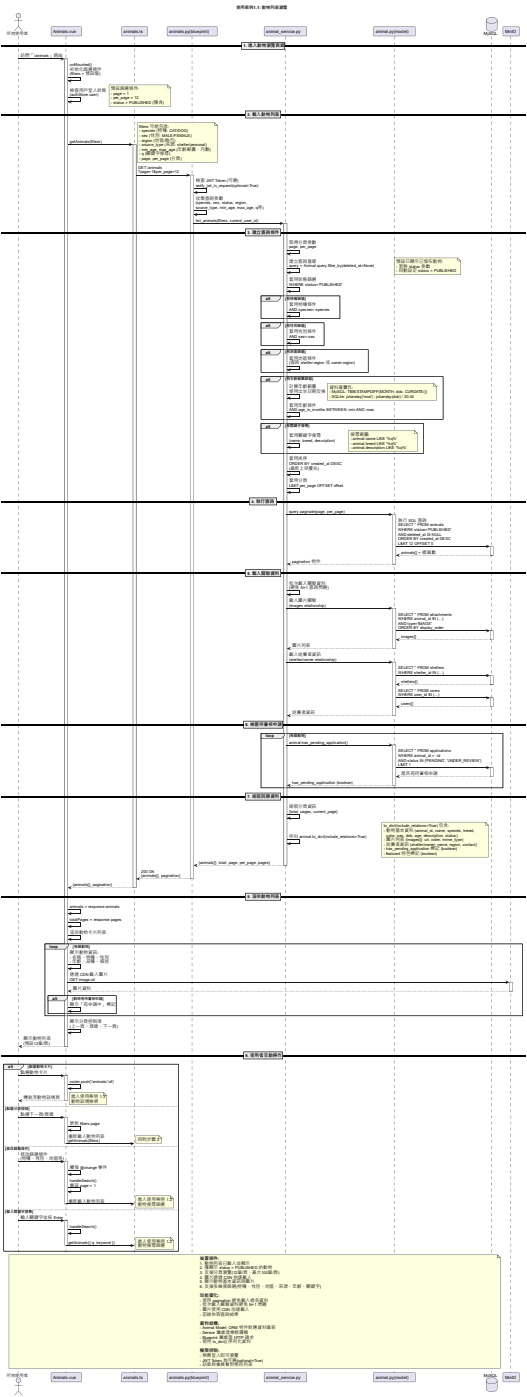
方法	端點	描述	認證	權限
GET	/admin/users	使用者管理列表	Yes	ADMIN
POST	/admin/users/:id/update-role	更新使用者角色	Yes	ADMIN
POST	/admin/users/:id/lock	鎖定使用者	Yes	ADMIN
POST	/admin/users/:id/unlock	解鎖使用者	Yes	ADMIN
GET	/admin/audit-logs	稽核日誌	Yes	ADMIN
GET	/admin/statistics	系統統計資料	Yes	ADMIN

總計: 60+ API 端點

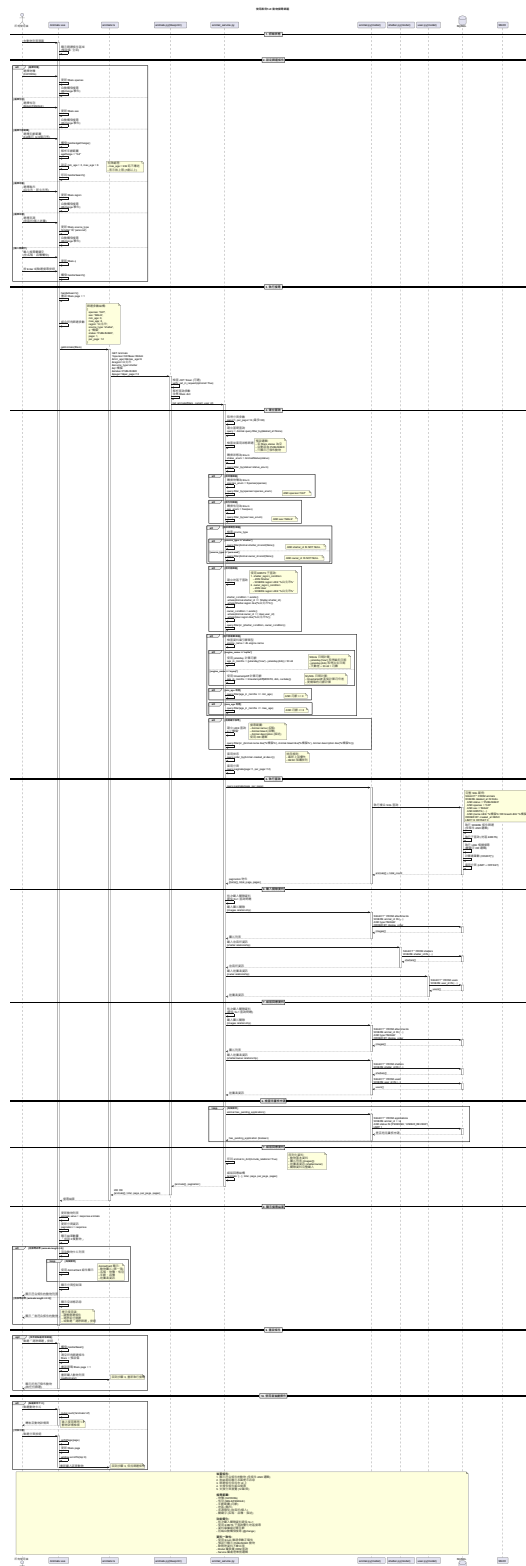
附錄 B: 循序圖完整列表

使用案例 1: 動物瀏覽與搜尋

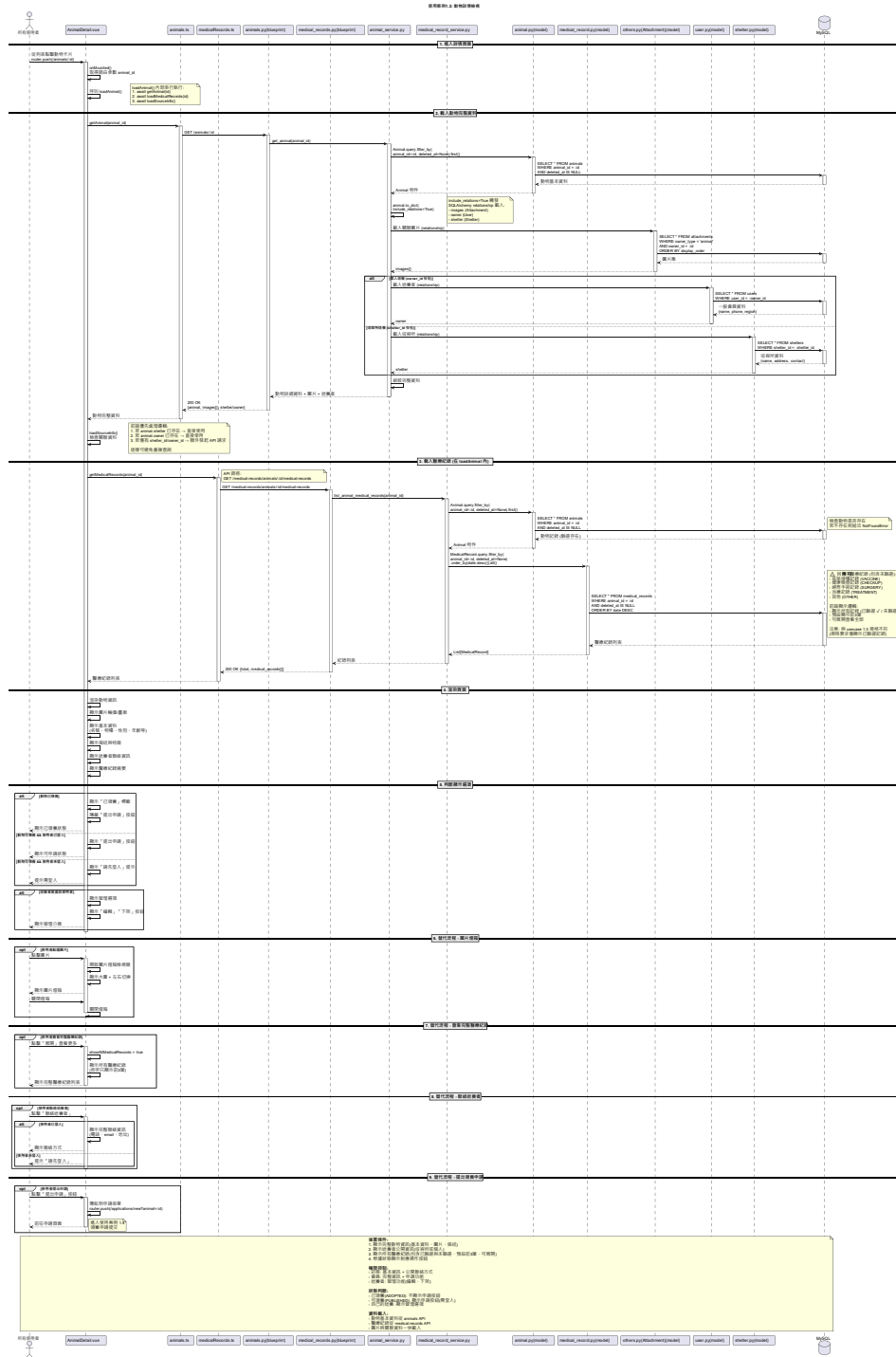
1_1: 動物列表瀏覽



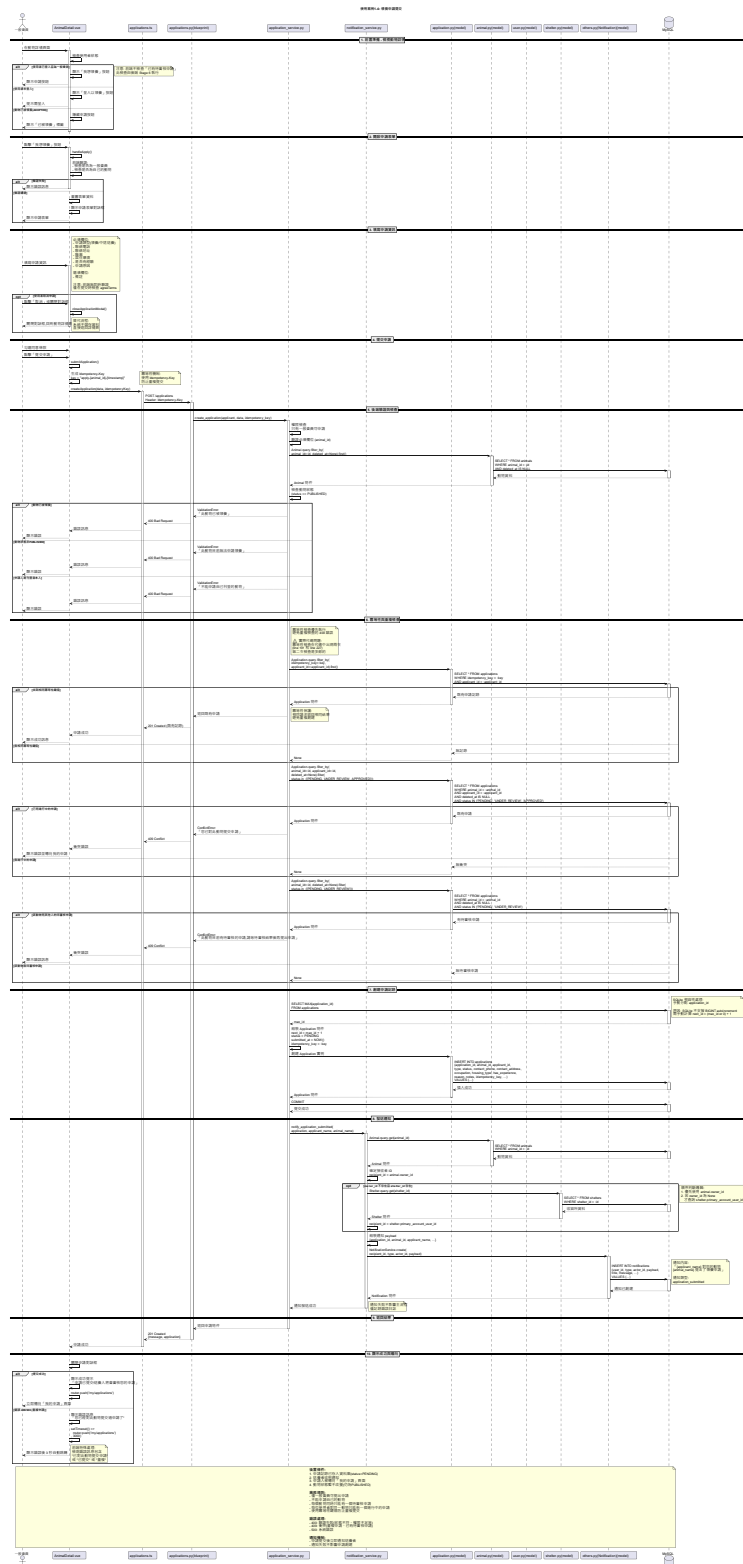
1_2: 動物搜尋篩選



1_3: 動物詳情檢視



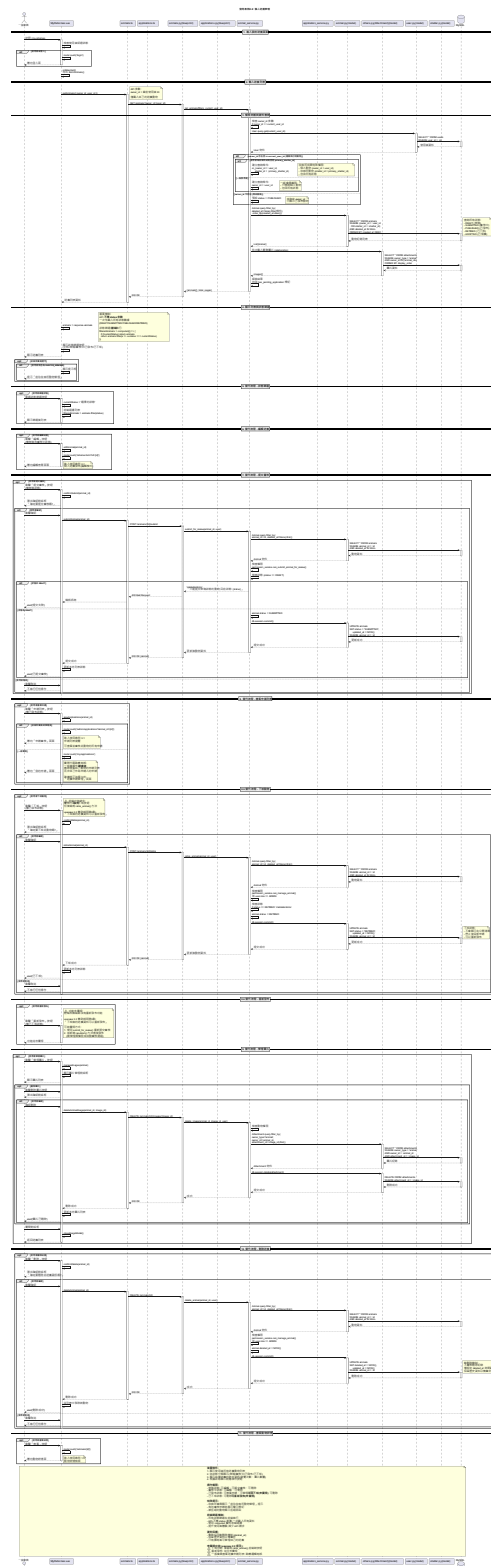
1_4: 領養申請提交



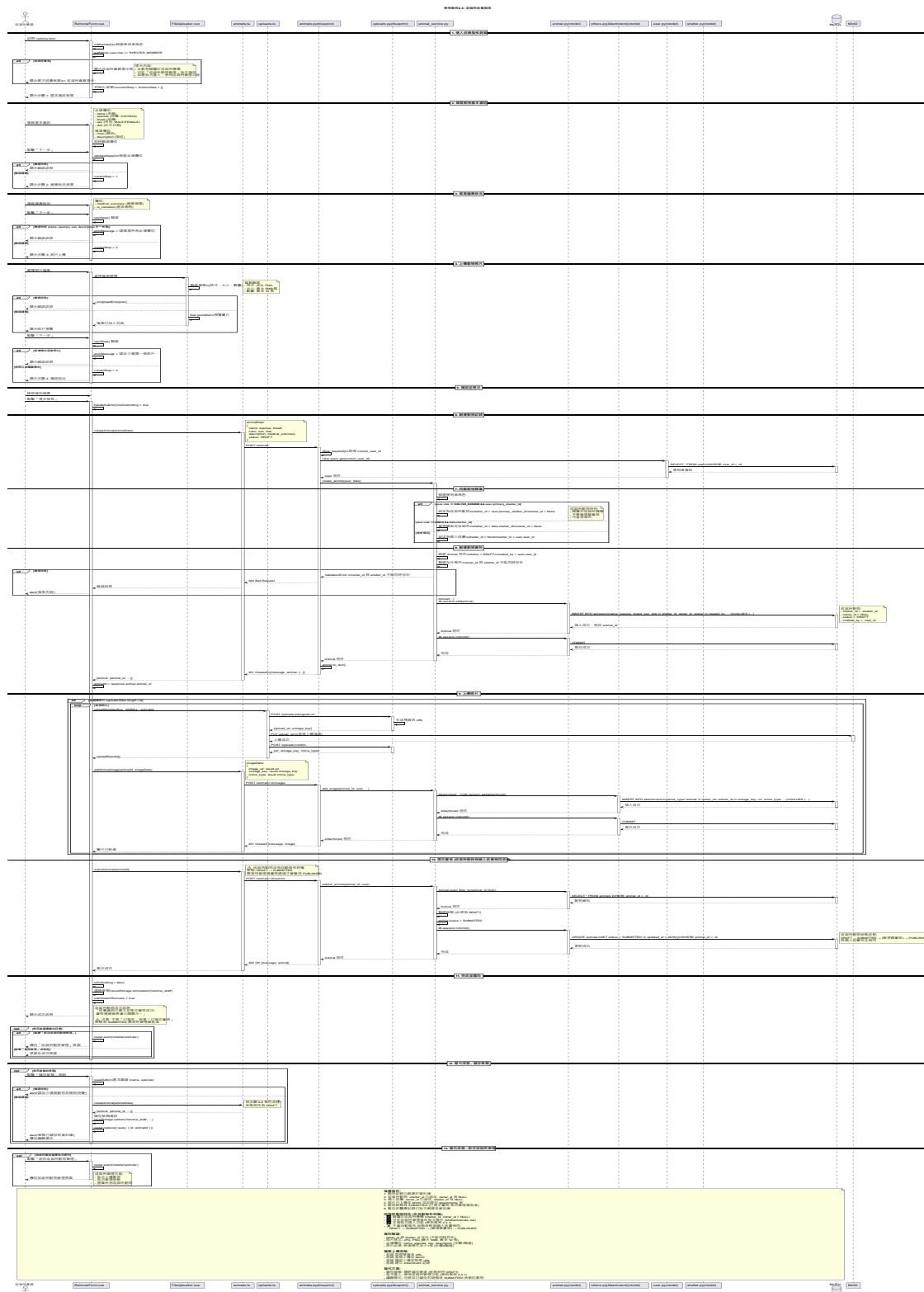
2_1: 個人送養發佈



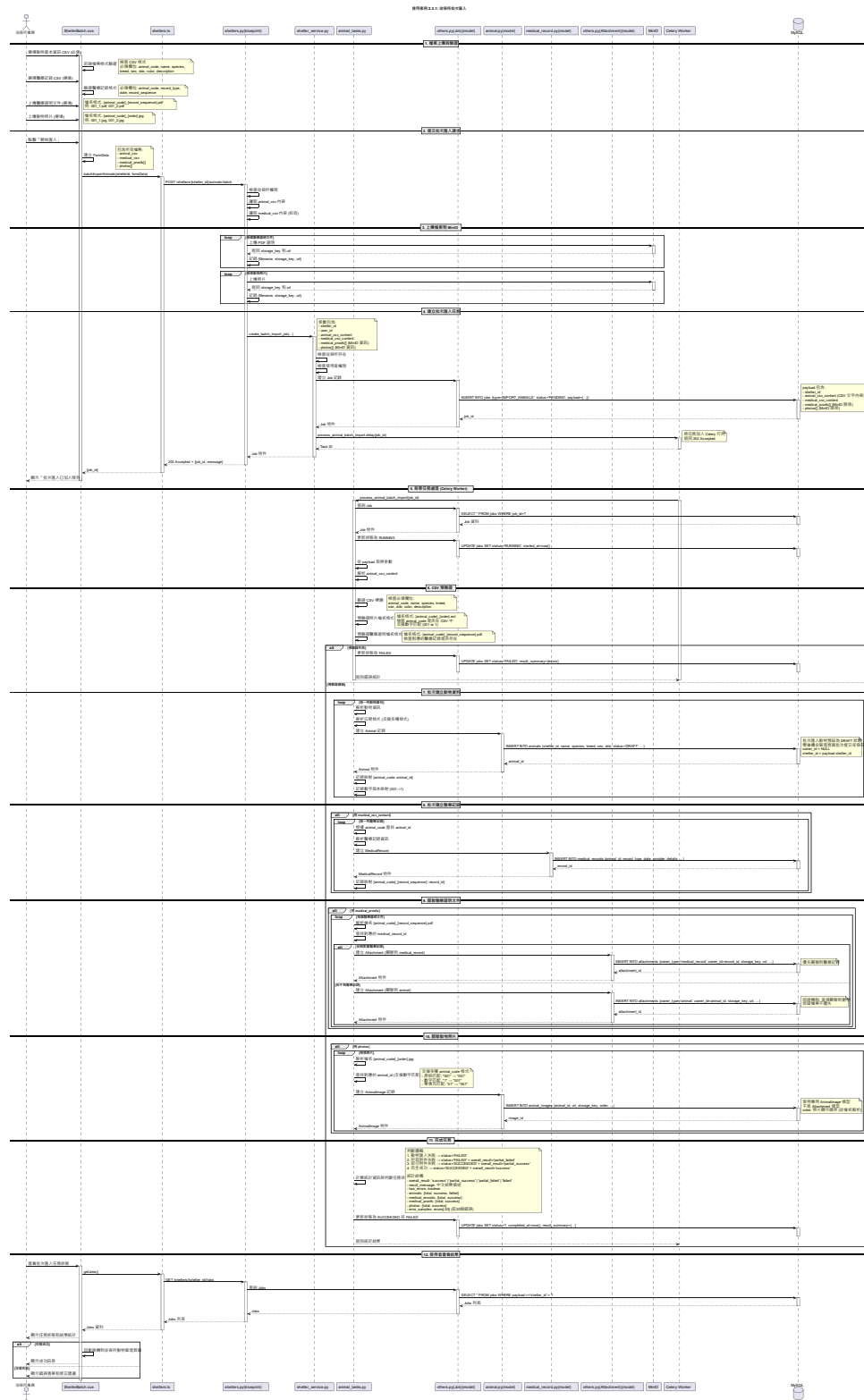
2_2: 個人送養管理



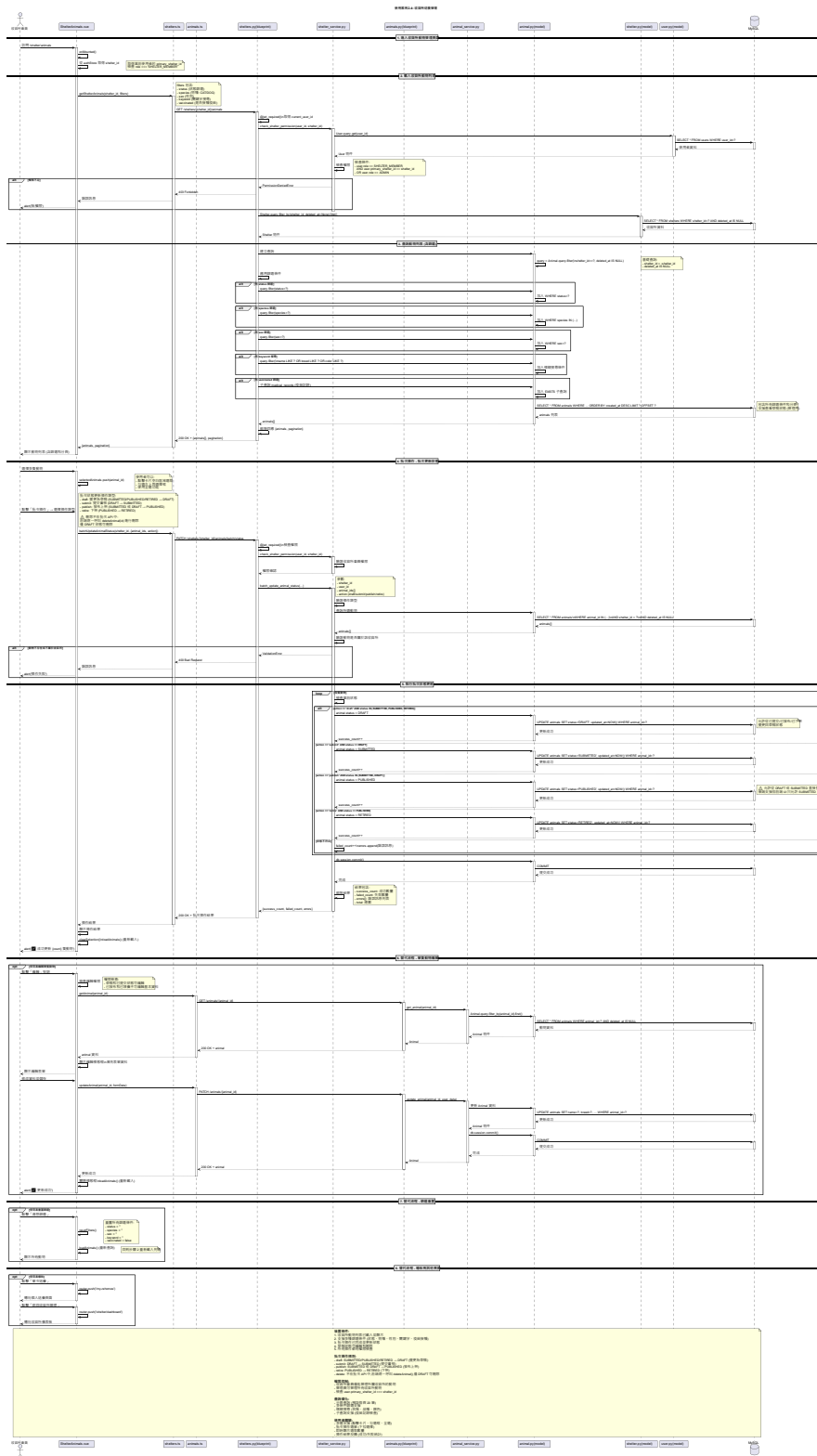
2_3: 收容所送養發布



2_3_1: 批次匯入



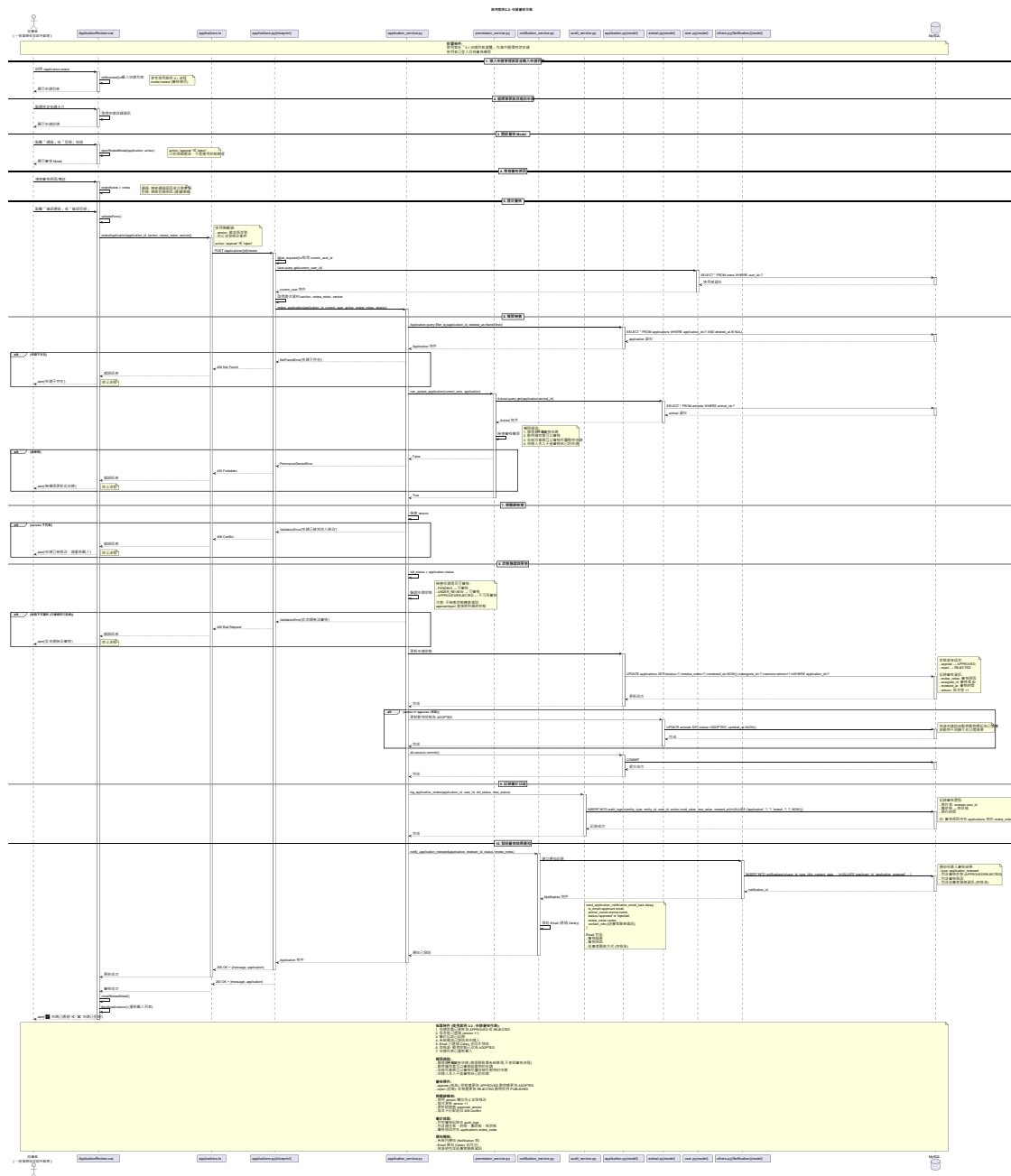
2_4: 收容所送養管理



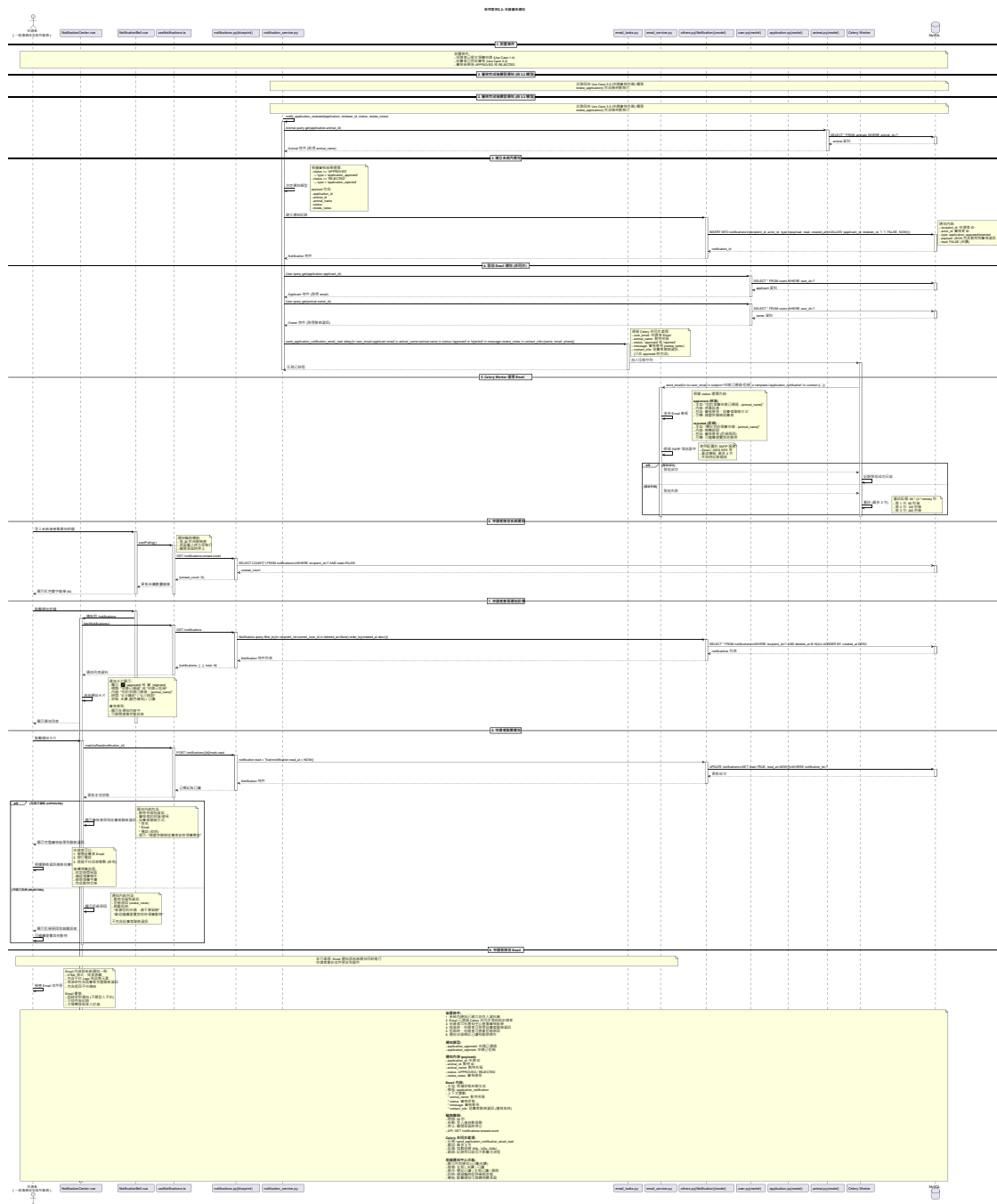
3_1: 申請列表瀏覽



3_2: 申請審核作業

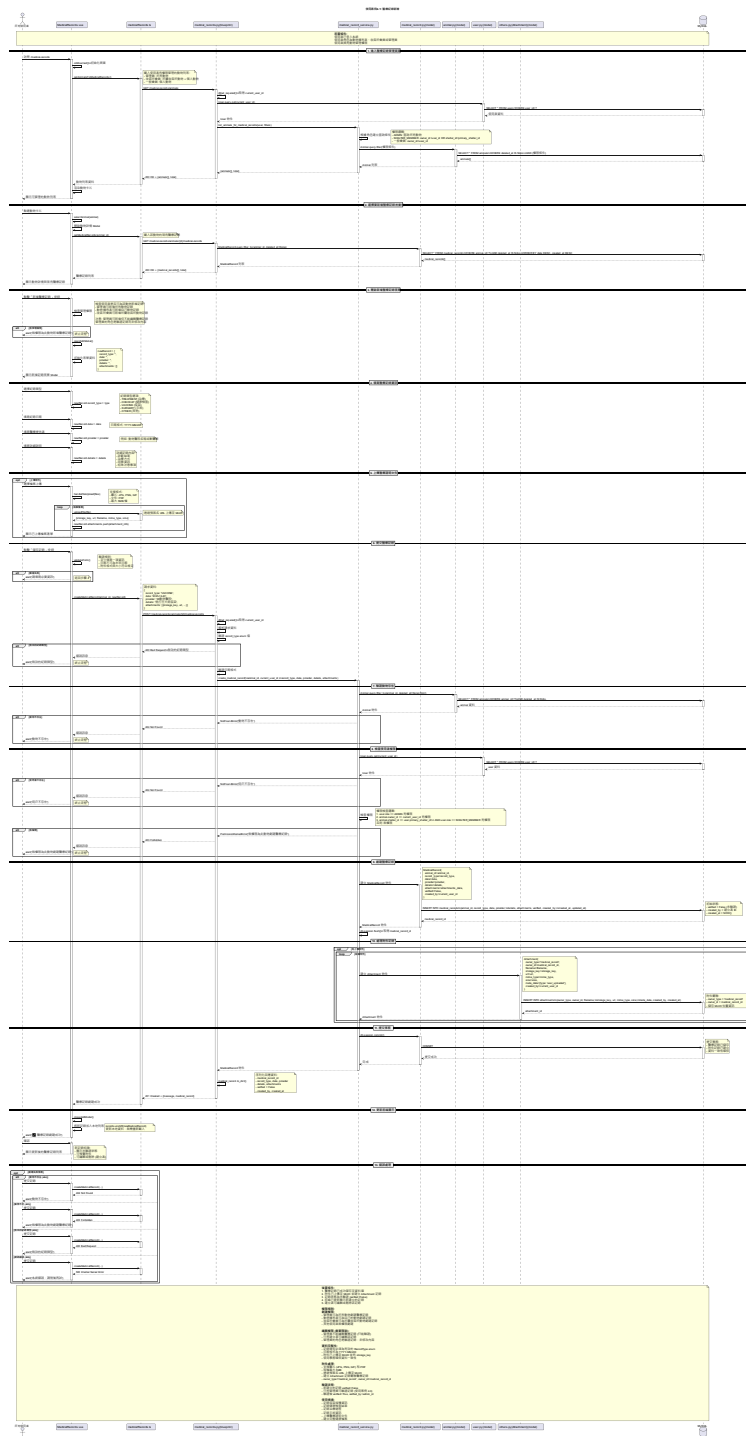


3_3: 申請審核通知

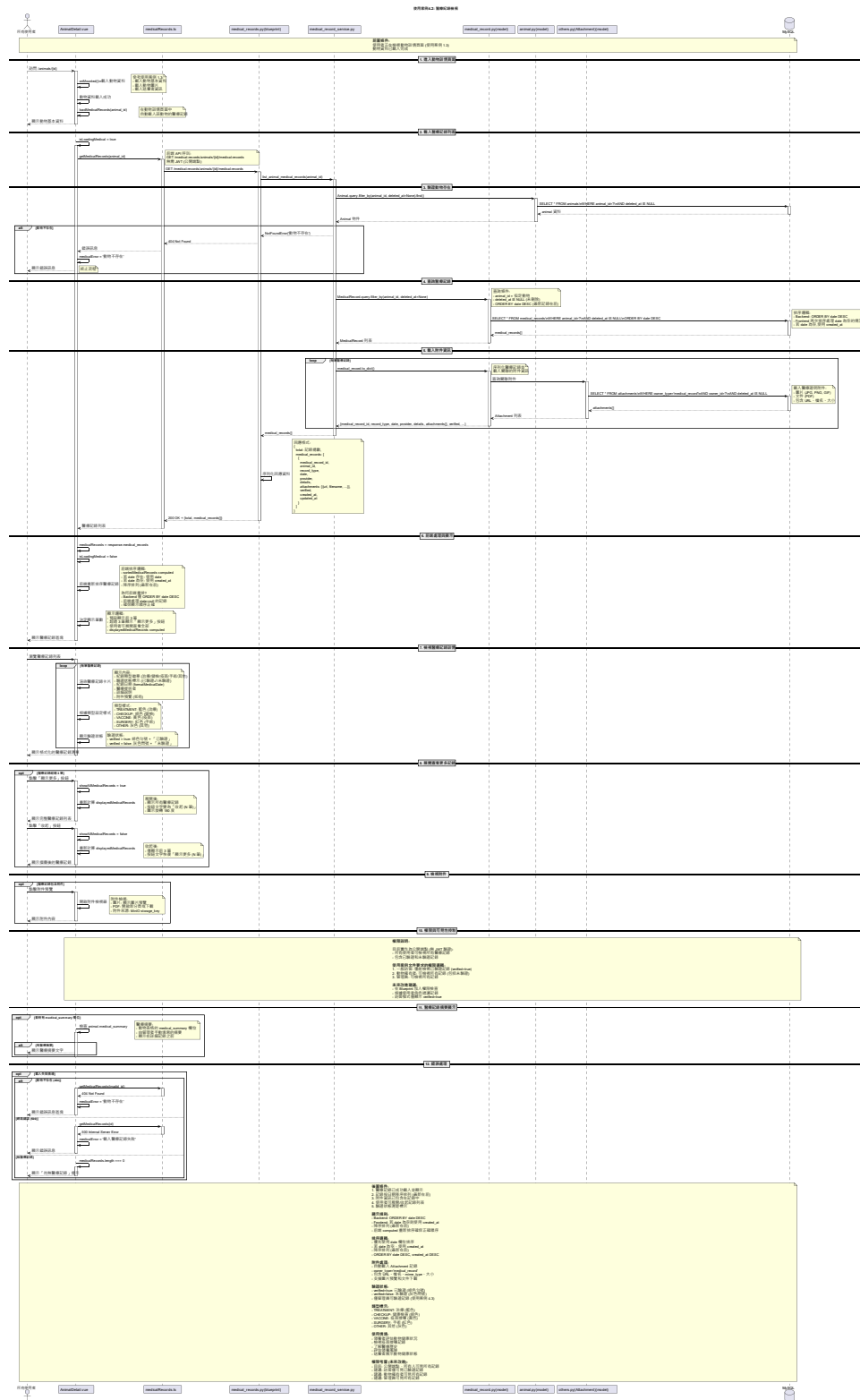


使用案例 4: 醫療紀錄

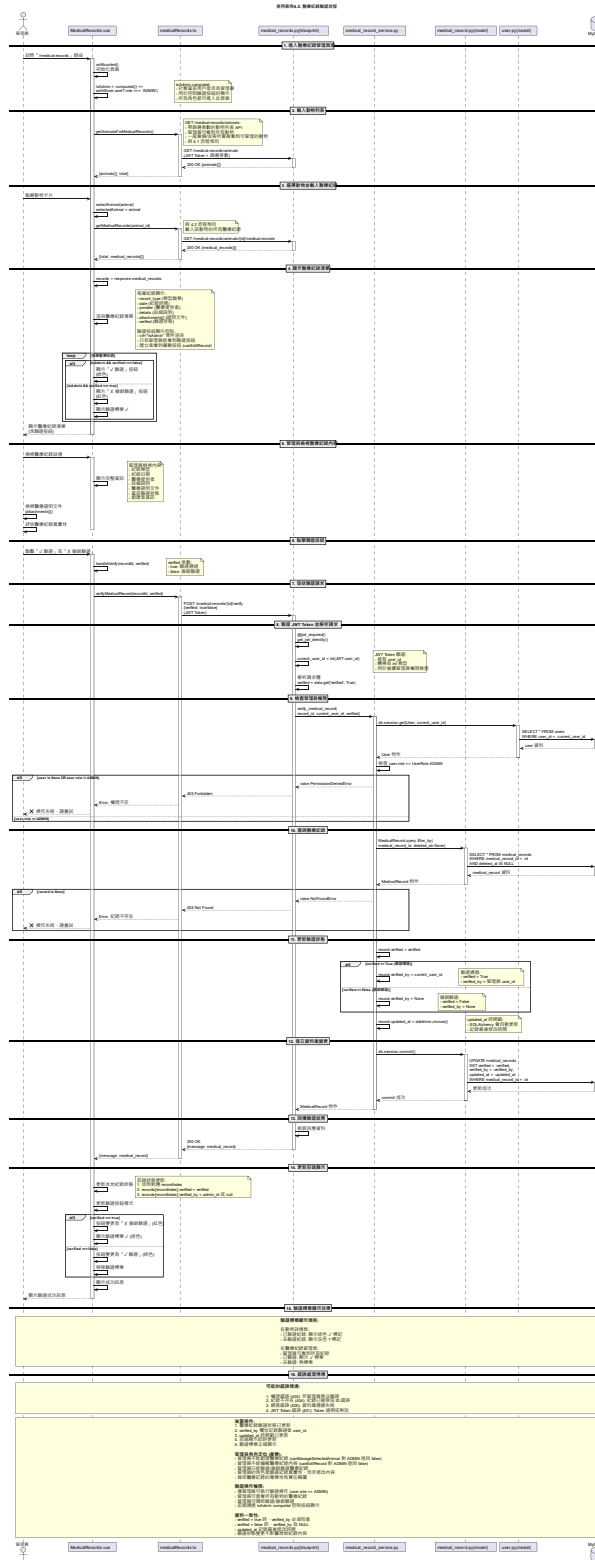
4_1: 醫療紀錄新增



4_2: 醫療紀錄檢視

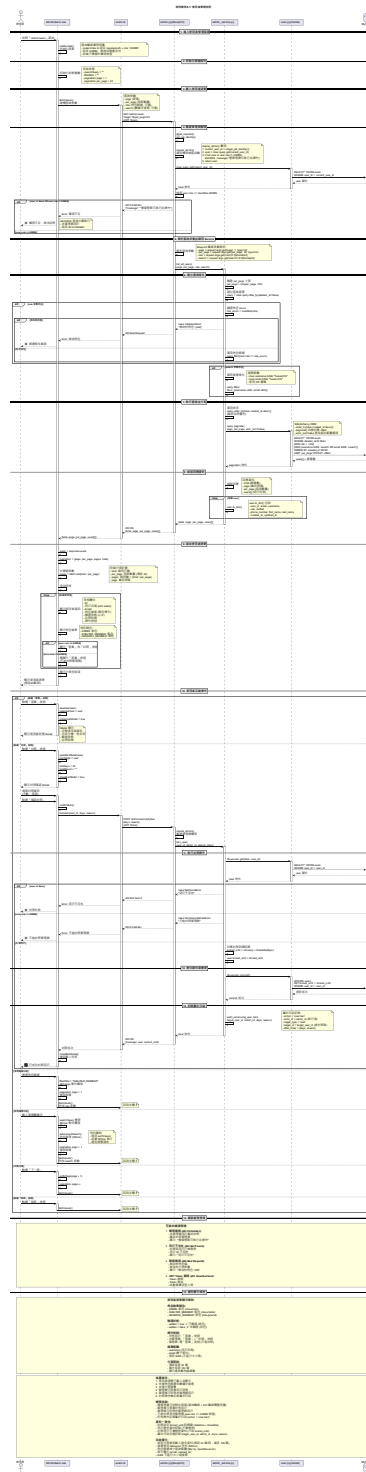


4_3: 醫療紀錄驗證

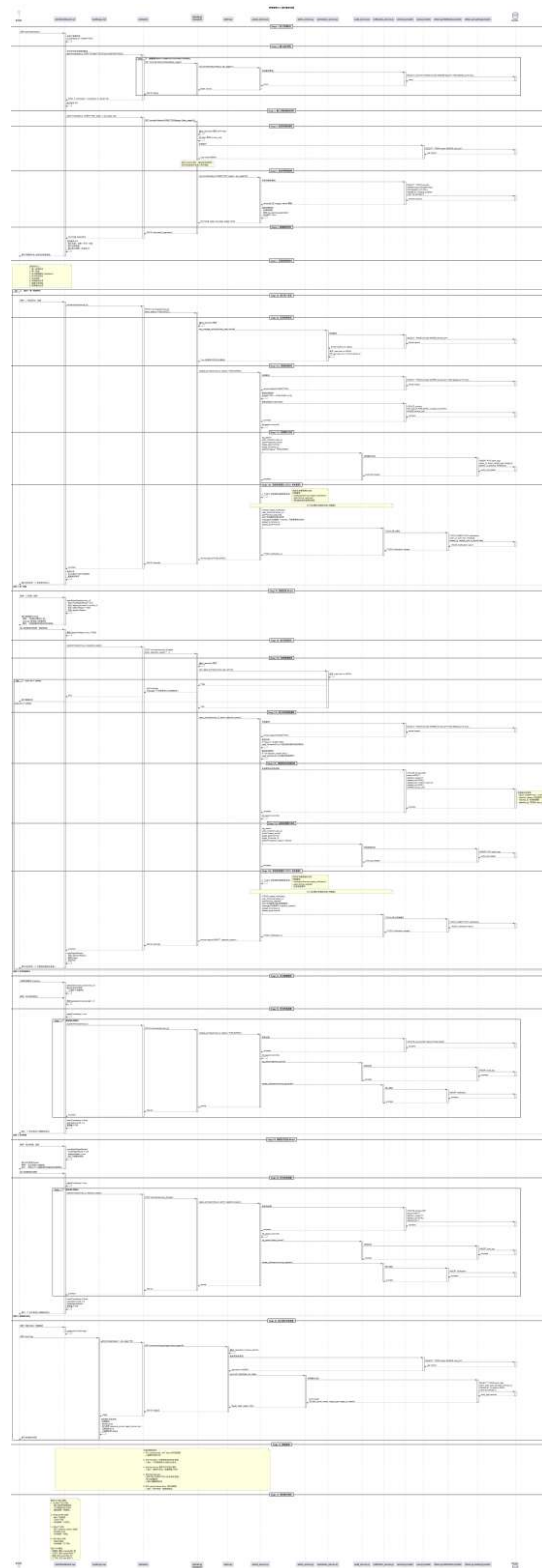


使用案例 5: 系統管理

5_1: 使用者管理

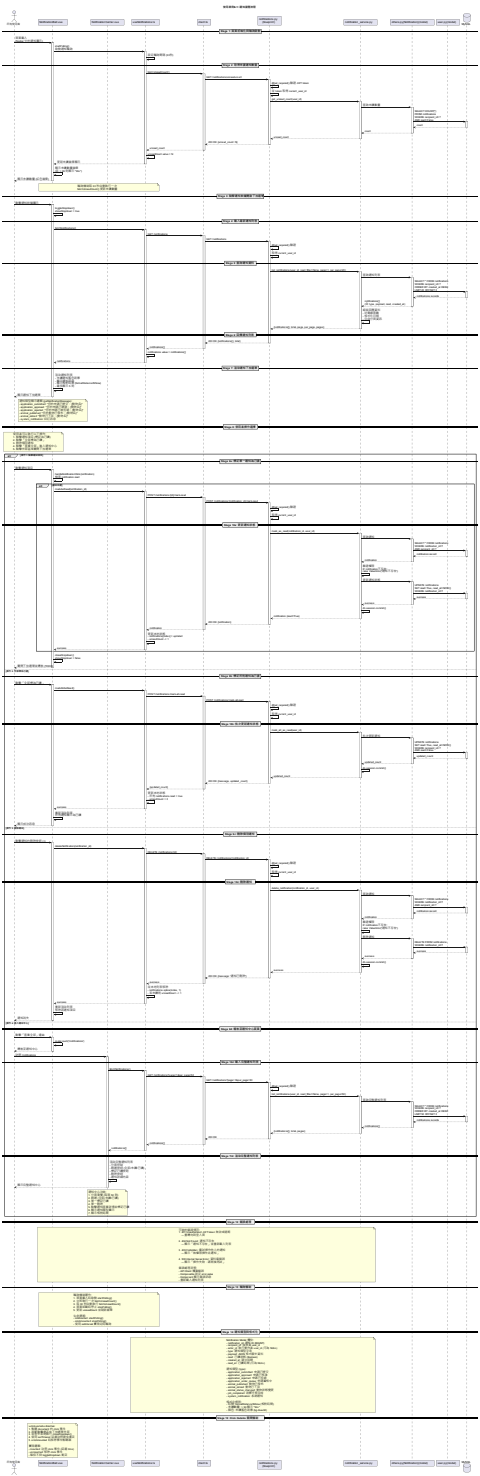


5_2: 資料審核

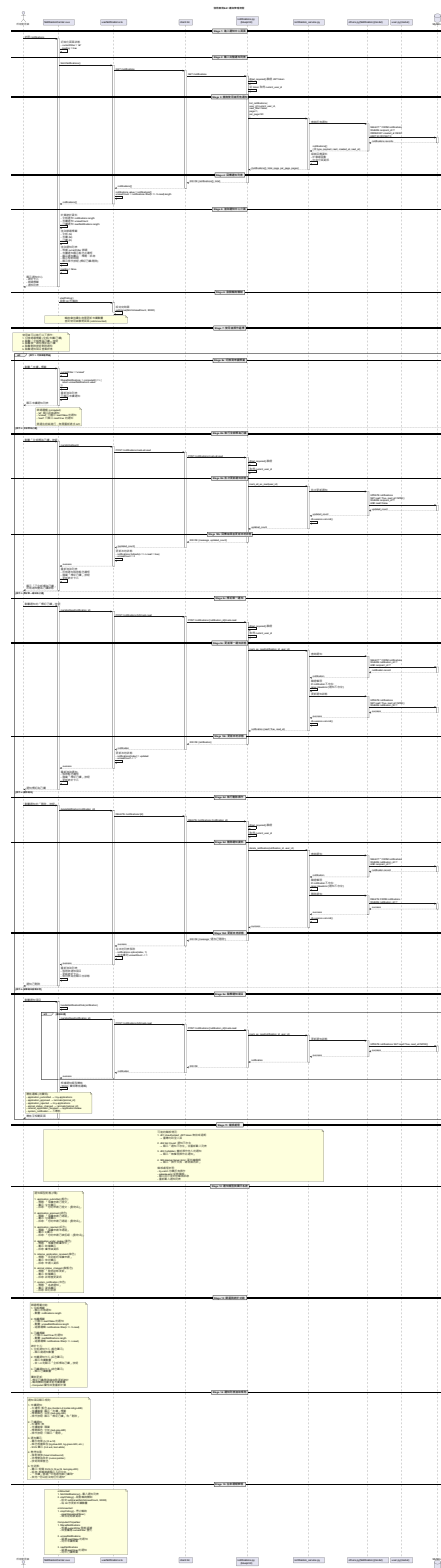


使用案例 6: 通知中心

6_1: 通知瀏覽



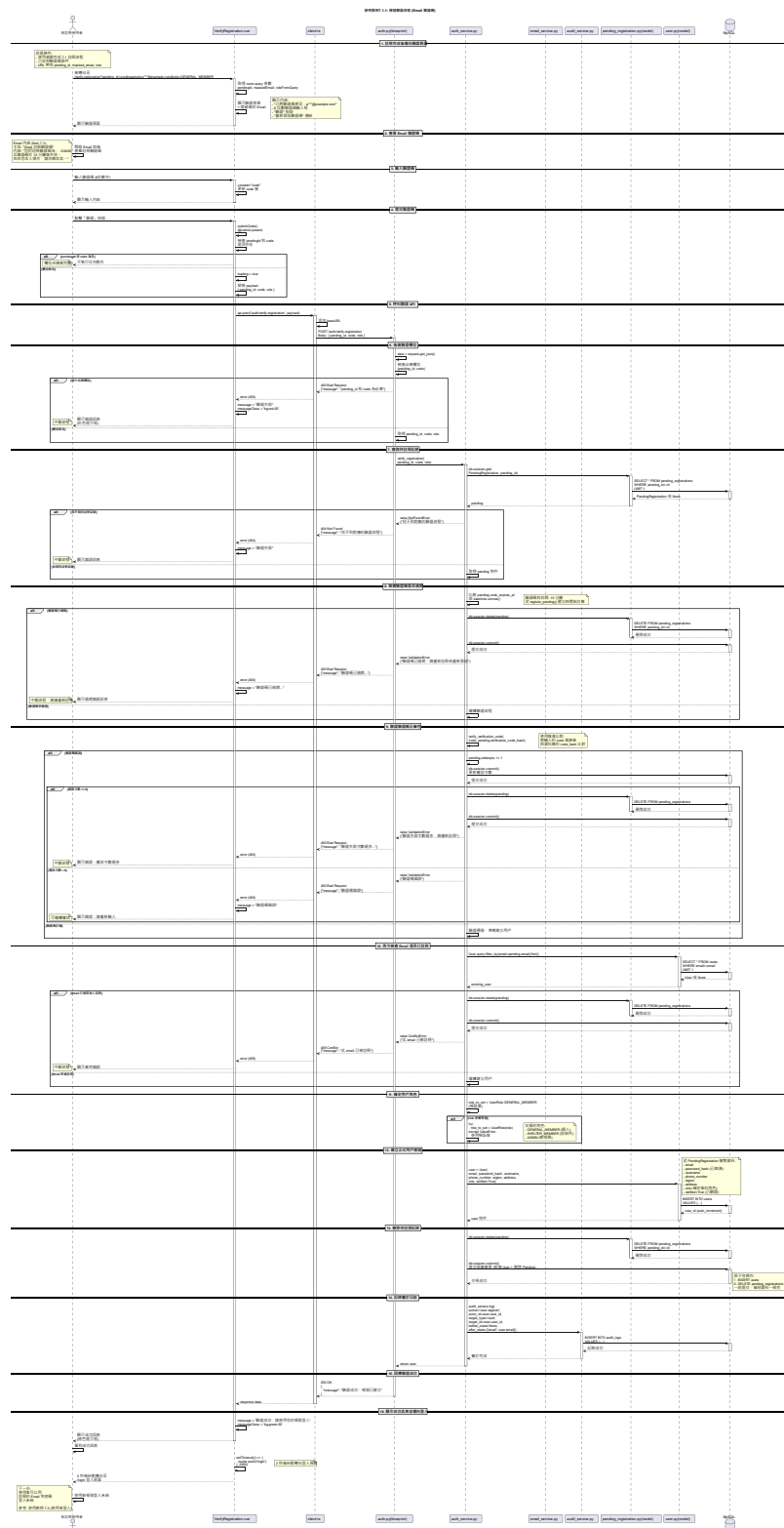
6_2: 通知管理



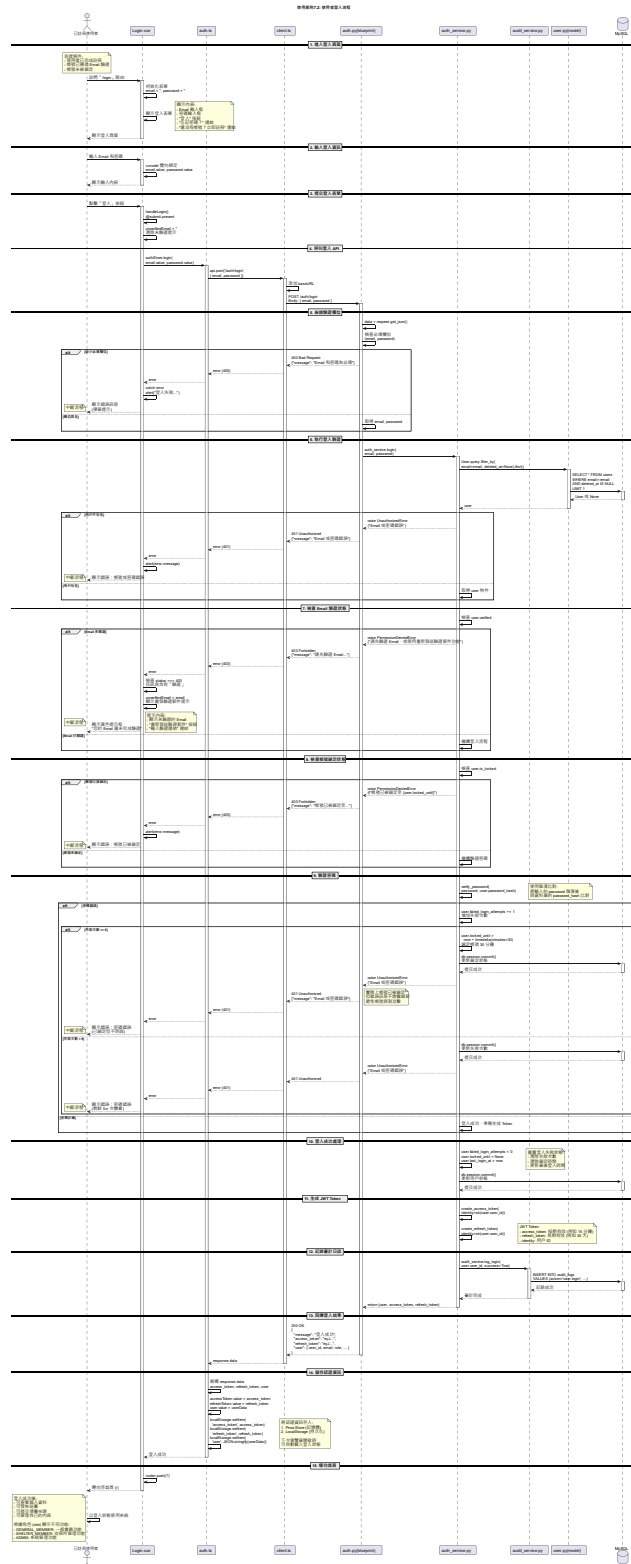
7_1: 使用者註冊



7_1_1: 帳號驗證



7_2: 使用者登入



7_3: 密碼管理

